Campus Shuttle Tracking & Demand Prediction

BCS 305 Software Architecture — Course Project Report

Canadian University Dubai · Fall 2026 · Instructor: Dr. Khaled Ammar

Team: **Abdullah** (GitHub `abdullahshammari796-ctrl`) · **Zuhair Halawa** (GitHub `zuhairhalawa`)

Student IDs: *added only in the private submission copy; not published in the public repository*

Version: technically verified candidate of 7 November 2026

> **Status of this document.** This is a technically verified candidate, not the final submission. The architecture, models, prototype and test evidence it describes exist in the repository and were verified between 31 October and 7 November 2026. Items marked **OUTSTANDING** need facts or actions that only the team or the instructor can provide: student IDs and signatures (private copy only), the proposal approval date, each member's own confirmation of the reflection, contribution and AI-use statements, and the team's own inspection of the diagrams on paper. Section 10 records the factual contribution and tool-use history; the personal statements in it must be completed truthfully by each member before submission.

1. Introduction and problem statement

The problem. Many UAE university campuses run shuttle buses between faculty buildings, student residences, car parks and the nearest metro station. Students cannot see where a shuttle is or when it will arrive, so they either wait at the stop — outdoors, in a climate where summer temperatures regularly exceed 40 °C — or miss the shuttle. The transport office, in turn, decides how many vehicles to run on which route from experience, because it has no record of where and when students actually wait.

Who has it and why it matters. *Students* lose time and comfort; students with reduced mobility are affected most. *Drivers* are interrupted by phone calls asking where they are. *Transport office staff* cannot see their fleet, cannot react when a vehicle stops reporting, and have no data to plan capacity. A system that shows live positions and honest arrival estimates, and that turns "I am waiting here" signals into demand history and forecasts, addresses all three.

Scope — included. Live shuttle tracking on a schematic campus map; arrival estimates per stop with an explicit confidence level; detection of delayed and lost GPS; trip management by drivers with a built-in GPS simulator; waiting requests; fleet and demand monitoring, route management and trip inspection by the transport office; historical demand trends and explainable demand forecasts with a model evaluation; role-based access, privacy controls and an audit log.

Scope — not included. Integration with a real university's single sign-on, student information system or vehicle telematics; native mobile apps and push notifications; timetable planning or automatic dispatching; payments; multiple campuses; Arabic localisation. All routes, stops, vehicles, accounts and historical demand in the prototype are **demonstration data**; nothing in this report describes the real operations of any university.

2. Stakeholders and requirements

2.1 Stakeholder analysis

Stakeholder	Interest / goal	Influence	What the system gives them	Main concern
Students (incl. students with reduced mobility)	Know when to go to the stop; spend less time waiting outdoors	Low individually, high collectively	Live map, ETAs with confidence, "I'm waiting here"	Accuracy; privacy (not being tracked)
Shuttle drivers	Do the job with minimal phone interaction; not be blamed for GPS faults	Medium	One-tap trip start/end, simulator for training, clear "what passengers see"	Distraction while driving; being monitored
Transport office staff (dispatchers)	See the fleet, react to incidents, plan capacity	High	Live operations view, remote trip termination, routes, trends, forecasts	Reliability of the data they act on
University IT / operations	Run the system securely and cheaply	High (approves deployment)	One process, one database file, no paid services	Maintainability, security, backups

Stakeholder	Interest / goal	Influence	What the system gives them	Main concern
Data-protection officer (UAE Federal Decree-Law 45/2021 on personal data)	Lawful, minimal processing of personal data	High (can block launch)	Data minimisation, pseudonymisation, retention limits, audit log	Location data of drivers; identities of students
University management (sponsor)	Better service at reasonable cost	High	Evidence (demand trends) for fleet decisions	Cost, adoption

2.2 Functional requirements

Status: ✅ implemented and verified · ◐ implemented with a stated limitation · ✗ not implemented.

ID	Requirement	Actor	Status
FR-01	Users sign in; every API call is authorised by role (student, driver, office)	All	✅
FR-02	Show active shuttles on a map of routes and stops, updated live	Student, Office	✅
FR-03	Show arrival estimates for a chosen stop, soonest first, with confidence	Student	✅
FR-04	Show route and stop information (routes serving a stop, description, waiting count)	Student	✅
FR-05	Register, replace and cancel a waiting request; see when it was served or expired	Student	✅
FR-06	Indicate delayed and lost GPS; never show an estimate without a position	Student, Office	✅
FR-07	Start a trip by choosing route and shuttle; end it (the trip ends on the device only after the server confirms)	Driver	✅
FR-08	Send location updates from device GPS or a simulator; buffer them when offline (bounded: see QA-4)	Driver	◐ device-GPS path implemented, not field-tested on a vehicle
FR-09	Show trip status, next stop and what passengers currently see	Driver	✅
FR-10	Report passenger load (seats available / filling up / full)	Driver	✅
FR-11	Monitor all active shuttles and their GPS state live	Office	✅
FR-12	See current waiting demand per stop live	Office	✅
FR-13	End a trip remotely with a mandatory, audited reason	Office	✅
FR-14	Manage routes: take in/out of service, edit attributes, create from stops	Office	◐ new routes use straight lines between stops
FR-15	Inspect trip history and the recorded GPS track of a trip	Office	✅
FR-16	View historical demand trends (per day, weekday × hour, per stop)	Office	✅ (synthetic + live data)

ID	Requirement	Actor	Status
FR-17	View explainable demand forecasts and compare models	Office	✅ (baseline models; see §6.7)
FR-18	Detect stale/lost GPS automatically; close trips silent for 30 minutes	System	✅
FR-19	Expire waiting requests after 20 minutes; mark them served when a shuttle arrives	System	✅
FR-20	Delete raw GPS after 30 days; erase waiting-request pseudonyms 24 h after closing	System	✅
FR-21	Record privileged office actions in an audit log	System	✅

2.3 Non-functional requirements as quality-attribute scenarios

Each scenario uses the six-part form (source, stimulus, artifact, environment, response, response measure). "Evidence" points to how it was verified; numbers are measured, not estimated. Full detail: `docs/architecture/quality-attribute-scenarios.md`.

ID	Quality	Scenario (source → stimulus → artifact, environment → response)	Response measure	Evidence (measured)
QA-1	Performance (freshness)	Driver app uploads a position → tracking + real-time gateway, normal operation → every connected viewer receives the new position and ETAs	p95 ≤ 1 s from upload to delivery	Local benchmark, 1,000 viewers: p95 **35–37 ms**, max **55–56 ms** (`npm run load-test`; same process, loopback, no browsers — see QA-1/QA-2 in `docs/architecture/quality-attribute-scenarios.md`)
QA-2	Scalability	3,000 students open the app at the start of term → one server → keeps delivering every update to everyone	max ≤ 1 s, no updates dropped for healthy clients	Local benchmark, 3,000 simulated viewers: max **108–113 ms**, 0 skipped. Not evidence for 3,000 real browsers on campus networks
QA-3	Availability of truthful information (GPS outage)	Shuttle GPS stops → tracking, trip active → viewers told "delayed", then "lost"; estimate hidden when lost; recovery on next position; trip auto-closed after 30 min	delayed within 22 s, lost within 62 s; 0 estimates shown in LOST; recovery on the first accepted fix	unit + integration tests; manual browser check
QA-4	Reliability (network loss)	Driver phone loses mobile data while GPS works, page stays open → driver app + tracking → positions are queued, delivered in order on reconnection; duplicates and out-of-order positions rejected	Bounded: no validated position lost for outages ≤ 10 min with the page open; beyond that, positions older than 10 min are rejected, the queue keeps the newest 500, and a page reload loses the queue	tests "shuffled offline backlog", "recovers when GPS resumes", uploader queue and end-trip tests; manual demo

ID	Quality	Scenario (source → stimulus → artifact, environment → response)	Response measure	Evidence (measured)
QA-5	Security (access control)	Unauthenticated caller or wrong role calls a protected endpoint → web API → request refused	100 % of 26 endpoints × 4 caller types answer 401/403; forged or expired tokens rejected; open live streams closed when the session expires; 5 failed logins → 5-min lockout	automated role-matrix test; token tests; live-stream expiry test
QA-6	Privacy	Any stored or displayed data → whole system → minimum personal data	students' location never collected; no driver identity in student-visible data; waiting pseudonym erased 24 h after closing; GPS traces deleted after 30 days	tests (view contents, retention job)
QA-7	Modifiability	Developer adds a new ETA algorithm or forecasting model → tracking / demand → added without touching services, API or UI	one new class + one line in the composition root	Strategy pattern (§7); model list in the UI is data-driven
QA-8	Testability	Developer runs the test suite → whole code base, developer PC, no network → all business rules exercised, time-dependent rules deterministic	full suite < 10 s	**119 tests in about 5 s** (`npm test`); 25 Chromium tests run separately
QA-9	Data consistency	Two drivers try to start trips on the same shuttle at the same moment → tracking + database → exactly one succeeds	100 %, guaranteed by the database	unique partial indexes + service check; tests

The **hard quality attribute** named in the proposal is *timeliness and honesty of shuttle information under unreliable GPS and mobile connectivity* (QA-1, QA-3, QA-4): the system must be fast when data is good, and must say so — instead of guessing — when it is not.

2.4 How the requirements were elicited and validated

Module 03 asks the report to say which elicitation techniques were used and what each taught. The project record shows three:

- **Document analysis.** The official project description, which lists "campus shuttle tracking and demand prediction" among its example themes, the Module 02 slides, the RBC Borealis ML guide and the UAE personal-data law (§2.1). It fixed the three user groups, the need for an explicit privacy treatment and the baseline-first approach to forecasting.

- **Domain analysis.** The stakeholder table, the functional requirements and the quality-attribute scenarios were drafted from general knowledge of transport-information systems while the baseline was prepared (see §10.2). This produced most of the requirements, but it is not evidence of what real students, drivers or office staff want.
- **Prototyping.** The running prototype, the GPS simulator, the load test and browser walkthroughs. Prototyping changed requirements: QA-4 was narrowed from "no position lost" to a bounded guarantee, the trip now ends on the driver's device only after the server confirms it, and the 6 November walkthrough showed that a driver page in a background tab stops reporting (§8.2).

No interviews, surveys or observations with real students, drivers or transport staff are recorded in the project evidence. If the team carried any out, they are **OUTSTANDING** and should be added here with what was learned. Requirements were validated against the running prototype (Appendix A.2 use cases, `docs/use-case-validation.md`), not with stakeholders.

3. Planning and development process

3.1 Process followed and why

We used a **risk-driven incremental process**: a deliberately chosen architectural skeleton first, then increments that each pass through analysis, design, build and test — with the riskiest questions answered by prototypes before the features that depend on them, in the spirit of the spiral model (BCS 305 Module 02). This "middle path" — enough architecture up front to fix the quality attributes and the component boundaries, then incremental delivery inside that structure — suited this project because:

- the quality attributes were known early (§2.3), and the expensive-to-reverse decisions (process structure, real-time transport, where ETAs are computed, how outages are represented) had to be made before building screens;
- a two-person team cannot absorb the coordination cost of a heavyweight process, but the course has fixed gates (proposal, report, presentation) that reward documented decisions;
- the demand-prediction part follows the ML lifecycle in the instructor's recommended reading (RBC Borealis field guide): discovery and feasibility first, a baseline before any model, and an explicit decision about whether data justifies more (§6.7);
- the V-model idea from the module was applied to testing: each specification level has a matching test level (§3.4).

Risk (what we did not know)	How it was reduced	Result
Can one ordinary server push updates to hundreds or thousands of viewers fast enough?	Prototype of the SSE gateway + local load test	3,000 simulated viewers in one process, max 108–113 ms; a deployment test with real clients is still needed
How do we stay honest when GPS or the network fails?	State machine for signal quality; store-and-forward on the phone	Delayed → lost → recovered, tested
Can ETAs be computed robustly on routes that use the same road both ways?	Linear referencing with a search window	Tested; the load test found and we fixed a corner-snapping bug
Is there data to justify machine learning?	Feasibility check (Borealis lifecycle)	No real data → explainable baselines + synthetic data clearly labelled

3.2 Increments

1. *Skeleton*: component structure, composition root, persistence, authentication, event bus.
2. *Tracking*: trips, location ingestion, route snapping, ETA strategies, signal state machine, live feed.
3. *Student and driver interfaces*, GPS simulator, offline queue.
4. *Demand*: waiting requests, demand history, predictors, backtest, office analytics.
5. *Hardening*: role matrix, privacy retention, audit log, load test, architecture-conformance test, documentation.

3.3 Actual timeline

The dated rows come from the repository, the pull requests and their reviews (times in Gulf Standard Time, UTC+4). Course deadlines come from the brief and the course schedule, whose dates are marked "indicative". What the project record does not show is marked **OUTSTANDING**; no date has been inferred from a commit time.

When	What happened	Evidence
Weeks 2–4 (11–25 Sep 2026)	Team formed and registered on Moodle (due Week 4); topic approved by the instructor (Week 4); one-page proposal due (Week 4)	**OUTSTANDING**: the actual dates of registration, proposal submission and approval are not in the project record. The proposal in `docs/01-proposal.md` was written on 31 Oct 2026
Week 6 (9 Oct) and Week 9 (30 Oct)	Model Set Studio sheet; Project Checkpoint 1 due (requirements, the four analysis models and the class diagram in one document)	**OUTSTANDING**: whether and what the team submitted is not recorded
31 Oct 2026	Increments 1–5 (§3.2) built and tested; first load tests	`docs/evidence/load-test-1000.txt`, `load-test-3000.txt`
1 Nov, 14:37–15:42	Public repository created; baseline imported (`8d11ad6`); work items opened as issues #1–#11	commits `8d11ad6`, `404238e`
1 Nov, 18:58–20:49	Zuhair's pull requests: #12 (fourth forecasting model, issue #6) and #13 (browser tests, accessibility and use-case fixes, ADR evidence review, CI workflow; issues #5, #7, #8, #9)	PRs #12, #13
1 Nov, 23:27–23:28	Owner review; #12 and #13 approved and merged; `main` protected (PR, 1 approval, `verify` check)	merge commits `d0775d3`, `7dcab45`; issue #4
1 Nov 23:45 – 2 Nov 00:22	Owner follow-up #14 (GPS-source resumption, waiting-refresh fix, dependency fix, UML corrections); approved by Zuhair at 00:05 and merged	review 5729974323; `f9327d4`
2 Nov, 00:45 – 23:01	#15: validated UML rendering and readable diagram pages. Zuhair requested changes at 01:05 (export validator accepted malformed files; A.3 layout); corrections pushed at 19:33; approved at 20:15; merged	reviews 5730257445, 5739614810; `5232999`
2 Nov, 23:06 – 23:53	#16: documentation follow-up; approved by Zuhair at 23:34; merged; `main` CI passed	review 5742229877; `695eb88`; CI run 41514123139

When	What happened	Evidence
6–7 Nov 2026	Final completion pass: requirement mapping, technical assessment, browser walkthroughs and a timed demo dry run, office accessibility fixes, UML-set print layout, retrospective draft, contribution and tool-use records, speaking script	branch `abdullah/final-completion`; review by Zuhair pending
Week 12 (20 Nov, planned)	Presentation running order published; any contribution concerns raised with the instructor	brief §2 and §4.4
Week 14 (4 Dec, planned)	Presentations day 1. The instructor's note on the course-introduction slides says every team submits everything before Day 1	course introduction, slide 11
Week 14 or 15 (4 or 11 Dec)	Presentation and live demonstration in the team's slot	**OUTSTANDING**: slot and time limit not yet published

3.4 Verification approach (V-model pairing)

Specification level	Matching test level	Where
Requirements (FR, QA scenarios)	Acceptance: scripted demo + browser walkthrough of UC1–UC5	`docs/demo-script.md`, `docs/test-plan.md`
System design (API, live feed)	System/integration tests over real HTTP and SSE	`test/integration/*.test.ts`
Architectural design (components, dependency rules)	Architecture-conformance test	`test/architecture.test.ts`
Module design (classes)	Unit tests	`test/unit/*.test.ts`

4. Analysis models

The full analysis models are in Appendix A: use case diagram (A.1), five fully dressed use case descriptions (A.2), activity diagram for *Run a trip* (A.3), system sequence diagram for *Run a trip and track it* (A.4), and state machines for the trip with its GPS signal (A.6) and for a waiting request (A.7). The primary use cases are:

- **UC1 Track shuttles and view arrival times** (Student)
- **UC2 Run a trip** — start, share location, end (Driver; GPS receiver as secondary actor)
- **UC3 Register a waiting request** (Student)
- **UC4 Monitor fleet and waiting demand**, extended by *End a trip remotely* (Office)
- **UC5 View demand forecast**, extended by *Explain a forecast* (Office)

The state machine for the trip is central to the design: an `Active` trip has the sub-states `AwaitingFix → Live ⇄ Stale → Lost`, driven by `after(20 s)` / `after(60 s)` time events and by accepted positions. The same four states exist as classes in the code (State pattern, §7.3), so the model and the implementation can be compared one-to-one.

5. Domain and structural models

Appendix A contains the domain model (A.8), two design class diagrams with attributes, operations, associations and multiplicities (A.9, A.10), the component diagram (A.11) and the deployment diagram (A.12).

Architectural views (4+1). Module 06 asks for the logical view and at least one other. All five are covered:

View	Question it answers	Diagrams
Logical	What the system knows and which classes carry the behaviour	A.8 domain model, A.9–A.10 class diagrams, A.13–A.17 patterns
Process	What happens at run time, in which order, and in which states	A.5 design sequence, A.6–A.7 state machines
Development	Which replaceable parts are built and what each provides and requires	A.11 component diagram
Physical	What runs where, and what happens when it fails	A.12 deployment diagram
Scenarios	What the system is for	A.1–A.2 use cases, A.3 activity, A.4 system sequence diagram

The hardest question for this system is "what do students see when a shuttle's GPS fails?", which lives in the process view (A.6 and the State pattern, A.15).

Domain model in brief. A *Route* visits two or more *Stops* in order (association class *RouteStop*); a *Trip* runs one *Route* with one *Shuttle* and one *Driver* and records many *LocationFixes*; a *Student* places *WaitingRequests* at a *Stop*; *HourlyDemand* observations at each stop are the input of *DemandForecasts*. Invariants: a shuttle and a driver are each on at most one active trip; an ended trip never changes.

Components (each is a folder under `src/`):

Component	Responsibility	Provided interface(s)	Required interface(s)
Web API (`web/`)	HTTP adapter: routing, input parsing, authentication and role checks, error mapping, security headers	REST API	application services of all components
Real-time Gateway (`realtime/`)	Server-Sent Events: snapshot on connect, coalesced pushes, heartbeats, slow-client handling; `LiveView` combines tracking and demand data	Live feed	`TrackingService`, `WaitingService` (read only), event subscription
Identity & Access (`identity/`)	Password hashing, signed session tokens, login throttling, pseudonymisation	`AuthService`	`UserRepository`
Network Management (`network/`)	Routes, stops, shuttles; route geometry (`RoutePath`)	`NetworkReader`, `NetworkService`	route/stop/shuttle repositories, events
Tracking (`tracking/`)	Trips, position ingestion, route snapping, ETAs, GPS signal states	`TrackingService` (incl. vehicle views)	`NetworkReader`, trip/fix repositories, events
Demand & Forecasting (`demand/`)	Waiting requests, demand history, predictors, backtest	`WaitingService`, `DemandService`	`NetworkReader`, waiting/history repositories, events

Component	Responsibility	Provided interface(s)	Required interface(s)
Event Bus (`shared/`)	In-process publish/subscribe of domain events	`EventPublisher`, `subscribe`	—
Persistence (`persistence/`)	SQLite schema, migrations, repository implementations	all repository ports	SQLite file

Deployment. For the prototype everything runs as one Node.js process on one machine; browsers connect over HTTP on `localhost`. For production we recommend a campus virtual machine behind a TLS reverse proxy (HTTPS is required for phone GPS and for `Secure` cookies) and daily backup of the database file (Figure A.12).

6. Architectural analysis and trade-offs

6.1 Architectural drivers

From §2.3, in priority order: (1) timeliness and honesty of shuttle information under unreliable GPS/connectivity (QA-1, QA-3, QA-4); (2) many concurrent viewers at peak times (QA-2); (3) security and privacy of location and identity data (QA-5, QA-6); (4) modifiability and testability, because forecasting and ETA methods will change as data accumulates (QA-7, QA-8); and two project constraints — a **two-person team** and **no paid services or cloud subscriptions** (the prototype must run on a laptop). The second constraint is the team's own; the brief does not require it.

The three requirements that drove the architecture (Module 03):

1. **QA-1, one update to every viewer within 1 s.** This is the only driver on which the chosen option (A) beats a polling monolith (C); it is why live data is pushed over Server-Sent Events (ADR-0002).
2. **QA-3, honest information when GPS stops.** It put ETAs and an explicit signal state machine on the server (ADR-0004), so every student sees the same state and "no estimate" instead of a guess.
3. **The two-person, no-paid-services constraint.** It ruled out microservices and a hosted backend in practice, and led to one deployable (ADR-0001) with an embedded database (ADR-0003).

6.2 Candidate architectures

A. Layered modular monolith with an in-process event bus and server push (SSE). One deployable. Inside, components with explicit port interfaces (hexagonal style); components react to each other through domain events; browsers receive pushes over Server-Sent Events.

B. Microservices. Separate Tracking, Demand, Identity and Notification services behind an API gateway, connected by a message broker (e.g. RabbitMQ or Kafka), each with its own database; WebSockets for clients.

C. Simple monolith with client polling. One MVC application; browsers poll `/live` every few seconds.

D. Serverless backend-as-a-service. A hosted real-time database and cloud functions (Firebase-style); clients subscribe directly to the hosted database.

6.3 Comparison against the quality attributes

Scores 1 (poor) – 5 (excellent) come from the technical assessment in `docs/architecture/technical-assessment.md`, which gives the reason for every cell. That assessment was produced with the tool declared in §10.2; it is not either member's personal scoring, which is still **OUTSTANDING** (issue #2). Only option A's freshness is measured (local benchmark) and option C's polling load is arithmetic; every other score is judgement. It replaces the draft of 1 November, which wrongly let the polling *transport* lower option C's modularity score and cited a "preference for no cloud subscriptions" that the brief does not contain.

Driver (weight)	A Modular monolith + SSE	B Microservices	C Monolith + polling	D Serverless BaaS	Justification
Freshness & fan-out, QA-1/2 (3)	5	4	2	4	A: p95 35–37 ms at 1,000 viewers in a local benchmark. B: broker and gateway add milliseconds, not seconds. C: data is on average half a polling interval old; meeting p95 ≤ 1 s needs about 1 s polling, i.e. 1,000–3,000 requests/s even when nothing moves (arithmetic). D: native push, but a cloud function per upload can add cold-start delays (not measured).
Robustness to GPS/network loss, QA-3/4 (3)	4	4	4	3	A, B and C can all run the signal states on the server and buffer positions on the phone; A is a single process but rebuilds live state after a restart (tested); B has more failure modes; polling (C) needs no reconnection logic. D: the 20 s / 60 s transitions happen when *no* event arrives, which needs client-side code or a scheduler, typically at one-minute granularity.
Security & privacy, QA-5/6 (2)	4	3	4	2	A/C: one trust boundary, data stays on campus. B: more network surface and service-to-service authentication. D: driver locations and student activity stored by a third party, possibly outside the UAE, which complicates data minimisation and cross-border rules.
Modifiability & testability, QA-7/8 (2)	4	3	4	3	A: ports, a composition root and an import-rule test; PR #12 added a forecasting model as one class plus one line. C: the transport does not decide module structure, so the same ports and tests are possible. B: cross-service changes need coordinated releases. D: logic split between client and vendor functions.
Operational simplicity for a team of two (3)	5	1	5	3	A/C: `npm install && npm start`, one database file. B: ≥ 5 deployables plus broker and gateway. D: no server, but vendor accounts, quotas and lock-in; offline demonstration is hard.

Driver (weight)	A Modular monolith + SSE	B Microservices	C Monolith + polling	D Serverless BaaS	Justification
Growth beyond one campus (1)	3	5	3	5	A scales vertically; the `EventPublisher` port is the seam for a shared broker. C's stateless servers scale out, at the cost of request volume. B and D scale per service or by the vendor.
Weighted total (max 70)	**61**	**44**	**52**	**45**	

Uncertainty. A and C differ only in freshness (15 against 6 points); without that driver both total 46. The choice of push over polling therefore rests entirely on QA-1's one-second requirement: if the office accepted information a few seconds old, C would be as good and simpler to scale out. A leads C by 9 points, so at least three one-point scoring errors in C's favour would be needed to tie. With equal weights the order is A 25, C 22, B 20, D 20; raising "growth" to weight 3 gives A 67, C 58, D 55, B 54.

6.4 Decision

We chose **A — a layered, modular monolith with an in-process event bus and Server-Sent Events**, the simplest architecture that meets every driver. It gives the best measured freshness, keeps all personal data in one place under one access-control layer, is operable by a two-person team, and keeps the evolution path open: each component already depends only on interfaces, so extracting the real-time gateway or replacing the event bus with a broker would not change business code. Microservices would buy independent scaling that one campus does not need, at a cost a two-person team cannot pay. The supporting decisions are recorded as architecture decision records:

ADR	Decision	Main alternative rejected
0001	Modular monolith with ports and adapters	Microservices; plain MVC
0002	Server-Sent Events for live updates; positions uploaded by REST	WebSockets; polling
0003	SQLite through Node's built-in `node:sqlite`, WAL mode	PostgreSQL server; in-memory only
0004	ETAs computed on the server; GPS freshness as an explicit state machine	ETAs in each browser
0005	Explainable seasonal baselines for demand; no trained ML model yet	Gradient boosting / neural model on synthetic data
0006	HttpOnly signed session cookies with role checks at the edge	JWT in localStorage
0007	Offline schematic SVG map instead of map tiles	Third-party map tiles
0008	TypeScript run directly by Node 24 (type stripping), no build step	Bundler + framework

6.5 Trade-offs, sensitivity points and risks

- **Single process (sensitivity point for availability).** If the process stops, live information stops. Mitigations: all trips and positions are persisted before memory is updated; on restart live state is rebuilt from the database (`TrackingService.recover`, tested); browsers reconnect automatically and receive a full snapshot. For a campus information service, a short outage during which students fall back to the printed timetable is acceptable; a lost record is not.

- **Synchronous SQLite calls (sensitivity point for latency).** Each statement blocks the event loop while it runs; at this data size that is short, but it depends on the query, the data volume and the disk. Fan-out was unaffected at 3,000 simulated viewers in the local benchmark; the risk returns if the database sits on a slow network drive.
- **In-process event bus (trade-off: simplicity vs horizontal scaling).** Fine for one instance. A second instance would need a shared broker behind the existing `EventPublisher` interface (ADR-0002).
- **SSE behind proxies.** Some proxies buffer streaming responses; the server sends `X-Accel-Buffering: no` and heartbeats every 15 s, and the client reconnects after 40 s of silence.
- **Device clocks.** Phones may have wrong clocks; every upload carries `sentAt`, and the server shifts all positions by the measured offset (tested with a 5-minute skew).
- **The driver page must stay in the foreground (risk found in the 6 November walkthrough).** In a background browser tab the simulated driver stopped sending after about five minutes, because browsers throttle timers in hidden pages; mobile browsers can likewise pause location in a page whose screen is off. The server then behaves as designed (delayed → lost), but the information is lost. A web page is therefore a demonstrator; a production driver client would need a native app or a dedicated in-vehicle device that can report in the background. This has not been tested on a phone (issue #3).

6.6 How the architecture addresses the cross-cutting concerns

Real-time communication and concurrent viewers. Drivers upload by ordinary REST; viewers receive Server-Sent Events. Each message is serialised once and written to every connection; all positions of one upload batch are coalesced into a single message per trip; a slow client whose socket buffer exceeds 256 KB skips incremental updates and receives a fresh snapshot when it drains, so one slow phone cannot grow server memory. On every (re)connection a snapshot replaces client state, so a client can never be left with a partial picture. A student page re-reads its own waiting request only after a snapshot, an arrival at its stop or a *fall* in that stop's waiting count, so students joining a busy stop do not make everyone else's page send requests.

ETA calculation. The route is *linear-referenced*: each GPS position is projected onto the route polyline to obtain "metres along the route". A search window around the previous progress keeps the shuttle on the correct lane of roads used in both directions; small backward jitter is ignored. ETA = remaining distance ÷ speed + dwell time at each intermediate stop. The default `BlendedSpeedEta` uses 60 % of the observed moving speed (exponentially smoothed, standing still ignored) and 40 % of the route's planned speed, clamped to 8–50 km/h, so one traffic jam or one GPS spike cannot produce absurd estimates. Estimates carry an absolute arrival time; browsers count down using a server-time offset, so device clocks do not distort them.

GPS outages and stale data. The `SignalState` machine (§7.3) classifies every active trip as awaiting fix, live, stale (> 20 s) or lost (> 60 s), measured from the newest accepted position in server-corrected time. Live estimates are shown as is; stale ones are labelled approximate and become "Due" when their time has passed; lost trips show **no** estimate, only "last seen". A delivered backlog whose newest position is already old is classified as stale immediately. Trips silent for 30 minutes are closed automatically, so no "ghost" shuttles remain. On the driver side, positions recorded during a mobile-network outage are queued in the open page and delivered on reconnection, within the bounds stated in QA-4 (newest 500 kept, at most 10 minutes old on arrival, lost if the page is closed). Ending a trip takes effect on the device only after the server confirms it. After a page reload, the driver app resumes the location source of the trip's latest accepted position; if none has been accepted yet, it shares nothing until the driver chooses, so a device-GPS trip never silently switches to simulated positions.

Persistence and data consistency. SQLite with write-ahead logging and foreign keys. Positions are written in one transaction per batch *before* in-memory state changes. Unique partial indexes (`… WHERE status = 'ACTIVE'`) make it impossible — even under concurrent requests — for a shuttle or a driver to have two active trips, and a student to have two open waiting requests. Positions are de-duplicated and ordered per trip; ended trips reject further positions (tested).

Security and role-based access. Passwords are salted scrypt hashes. Sessions are HMAC-signed tokens in `HttpOnly`, `SameSite=Strict` cookies (8-hour expiry), checked on every request together with the account's existence. A live SSE stream is authorised when it opens and its session's expiry is re-checked before every push and at every 15 s heartbeat; an expired stream receives `session-expired` and is closed. Tokens are stateless, so sessions **expire** but cannot be **revoked** early: logout deletes the cookie in that browser only, and there is no server-side revocation list (a known prototype limitation). Roles are enforced at the edge by route prefix (`/api/student`, `/api/driver`, `/api/office`), and ownership inside services (a driver can only post to their own trip). State-changing calls require a custom header (CSRF defence in depth). A strict Content-Security-Policy with no inline scripts or styles, `X-Frame-Options: DENY`, `nosniff`, no-referrer and `no-store` on API responses are sent; all user-visible text is HTML-escaped. Five failed logins lock a username for five minutes. Office actions are audited.

Privacy. Data minimisation by design: students never send their location (they choose a stop); student-visible data never contains driver identity; waiting requests are linked to students only through a keyed pseudonym that is erased 24 h after the request closes; drivers' positions are collected only during an active trip and raw traces are deleted after 30 days; the map uses no third-party tiles, so viewers' locations are not disclosed to anyone. Details and a data inventory: `docs/privacy-and-security.md`.

Performance and scalability. A local benchmark on a Windows 11 desktop PC (Intel Core i7-14700F, 16 GB RAM, Node 24.14.1) gave: 1,000 simulated viewers p95 35–37 ms, max 55–56 ms; 3,000 viewers p95 90–98 ms, max 108–113 ms; process memory 311–332 MB including the 3,000 test clients. The viewers ran in the same process as the server over loopback, shared one demo session, and rendered nothing, so these numbers describe the server's fan-out, not the experience of 3,000 real students on campus Wi-Fi (method and remaining uncertainties: `docs/architecture/quality-attribute-scenarios.md`). The scaling path is vertical first, then a broker behind `EventPublisher` and several instances behind a load balancer.

Maintainability and testability. Constructor injection everywhere; the composition root (`src/container.ts`) is the only place that knows concrete classes. A `Clock` port makes every time rule deterministic in tests. `test/architecture.test.ts` enforces the component diagram's dependency rules on the code's imports: tracking and demand never import each other; business components see the network component only through types; only the composition root and seeding know persistence classes. It checks imports only, not the operations, associations or multiplicities in the class diagrams; those were compared with the code by hand (1 November 2026).

6.7 Demand prediction and the ML development lifecycle

The instructor's recommended reading (RBC Borealis, *Field Guide to Machine Learning Product Development*) describes a lifecycle of **discovery → feasibility → design requirements → model and product development → production → support and maintenance**, and stresses starting from a baseline and checking whether ML is needed at all. We applied it as follows:

Lifecycle stage	What we did
Discovery	Problem framed as "how many students will wait at each stop in each hour", to help the office allocate shuttles. Data sources identified: waiting requests and trips recorded by this system.

Lifecycle stage	What we did
Feasibility	No real historical data exists for any campus. A trained model would have nothing real to learn from, and its accuracy could not be meaningfully evaluated. Decision: build **explainable baselines** and the full pipeline; use **synthetic data, clearly labelled**, only to demonstrate and test it.
Design requirements	Office staff must be able to see *why* a number is shown and how uncertain it is: every forecast lists its past observations, weights and a low–high range; "no history" is shown instead of a guess.
Model & product development	Four strategies behind one interface: *seasonal average* (same weekday and hour, 4 weeks), *weighted seasonal* (6 weeks, recent weeks weigh more), a robust *same-weekday median* (6 weeks) and a naive *hour-of-day mean* that a useful model must beat. A rolling-origin backtest computes MAE, RMSE, bias and range coverage without look-ahead (tested).
Production	Forecasts are computed on request from the live database; every live waiting request automatically becomes training/evaluation data.
Support & maintenance	The backtest table is the monitoring tool: when enough real data exists (we suggest at least one full semester), rerun the evaluation on live data only; introduce a trained model only if it beats the seasonal baseline on held-out real weeks.

Backtest on the deterministic synthetic history (14 days, 1,890 stop-hours): seasonal average MAE 1.45, weighted seasonal 1.44, same-weekday median 1.42, hour-of-day mean 1.98 requests per stop-hour. The full `npm run evaluate` command was reproduced on Linux with Node 24.22.1 in Zuhair's verification on 1 November 2026, and again on Windows 11 with Node 24.14.1 during the owner review. **These numbers only show that the pipeline works and that weekday seasonality matters in data generated with weekday seasonality; they are not evidence of real-world accuracy.** Full description, assumptions of the generator and limitations: `docs/demand-prediction.md`.

7. Design patterns applied

Five patterns are applied; each has a class diagram in Appendix A (A.13–A.17) and is visible in the named source files.

7.1 Strategy — ETA estimation and demand forecasting (Figure A.13)

Problem. ETA and forecasting methods must change as we learn (e.g. a trained model later) and must be compared side by side, without editing the services that use them. *Solution.* `EtaStrategy` (`NominalSpeedEta`, `BlendedSpeedEta`) is injected into `TrackingService`; `DemandPredictor` (`SeasonalAveragePredictor`, `WeightedSeasonalPredictor`, `SameWeekdayMedianPredictor`, `HourOfDayMeanPredictor`) implementations are injected into `DemandService`, which also backtests all of them. *Consequence.* QA-7 is met: a new algorithm is a new class plus one line in `src/container.ts`; the office UI lists models from the API. *Code:* `src/tracking/eta/`, `src/demand/prediction/`.

7.2 Observer (publish/subscribe) — domain events (Figure A.14)

Problem. A shuttle update must reach many independent receivers — hundreds of browsers and the waiting-request module — without the tracking module knowing any of them; tracking and demand must stay decoupled. *Solution.* `EventBus<DomainEvent>` with typed subscriptions. Observers are **callbacks**, not classes implementing an interface: a handler has the function type `EventHandler<E> = (event: E) => void`; `SseHub` registers arrow functions in its constructor and the composition root registers a lambda that calls `WaitingService.markServed` on `stop.arrived`. Publishers depend only on the `EventPublisher` interface. Each

handler runs inside `try/catch`, so one failure cannot break the publisher. *Consequence.* New reactions (e.g. notifications) need no change in tracking; the bus is the seam for a future broker. *Code:* `src/shared/event-bus.ts`, `src/realtime/sse-hub.ts`, `src/container.ts`.

7.3 State — GPS signal of an active trip (Figure A.15)

Problem. What passengers may be told depends on the freshness of GPS data, and the rules for reacting to new positions and to time passing differ per situation. *Solution.* `SignalState` with `AwaitingFixState`, `LiveState`, `StaleState`, `LostState`; the `TripTracker` context delegates `onFix`, `onTick` and `present(etas)` to its current state. States are stateless singletons. *Consequence.* No status `switch` statements spread across the code; the UML state machine (A.6) maps one-to-one onto classes. *Code:* `src/tracking/signal-state.ts`, `src/tracking/trip-tracker.ts`.

7.4 Repository (ports and adapters) — persistence (Figure A.16)

Problem. Business rules must not depend on SQL and must be testable without a database. *Solution.* Each component declares repository interfaces (`TripRepository`, `LocationFixRepository`, `WaitingRequestRepository`, …); `src/persistence/` implements them for SQLite, and in-memory versions exist for tests. *Consequence.* `TrackingService` is unit-tested with in-memory repositories; the database could be replaced by PostgreSQL by writing new adapters only. *Code:* `src/*/ports.ts`, `src/persistence/`.

7.5 Adapter — interchangeable GPS sources (Figure A.17)

Problem. The system must be demonstrable without vehicles, yet the demonstration must exercise exactly the same upload path as a real phone. *Solution.* `DeviceGps` adapts the browser Geolocation API to a small `start(onFix, onError) / stop()` contract; `SimulatedGps` implements the same contract by generating positions from route geometry. Because the driver app is plain JavaScript, `GpsSource` is a **conceptual (duck-typed) contract** documented in `gps.js`, not a declared interface; Figure A.17 labels it so. The trip-running code uses only `start` / `stop`; the demo control panel also calls simulator-only operations (speed, outage, glitch). Positions flow from the `onFix` callback into the `LocationUploader`. *Consequence.* The server processes both sources identically. It stores each position's `source` only for the record and to tell a reloaded driver page which source to resume, so a device-GPS trip never continues with simulated positions. *Code:* `public/js/gps.js`.

Supporting design techniques: *Dependency Injection* with a *Composition Root* (`src/container.ts`), *Facade*-like application services per component, and *Linear Referencing* for route geometry.

8. Prototype

8.1 What was built

A complete, locally runnable web application (`npm install`, `npm start`, open `http://localhost:3000`):

- **Student**: live map with route filter; stop selection (remembered per device); next arrivals with minutes, clock time, distance, load and signal badge; delayed/lost indicators; "I'm waiting here" with route option, cancel, served/expired feedback; connection banner.
- **Driver**: start trip (route, shuttle, GPS source); live trip card with next stop and ETA, GPS and upload status, "what passengers see"; passenger load; end trip with confirmation; resume after reload with the same location source; GPS simulator with speed control, GPS-outage, network-loss and glitch-injection buttons; activity log.

- **Transport office**: KPI tiles, fleet map with waiting badges, waiting per stop, active-trip table with remote termination; demand forecast (model and date), hourly chart, stop × hour heat map with per-cell explanation, historical trends, model backtest; route management (service status, attributes, new routes); trip history with GPS track and gap statistics; system health, privacy/retention summary and audit log.
- **Tooling**: deterministic demo seed, headless fleet simulator (`npm run simulate`), load test, demand evaluation, 119 Node tests (unit, integration, an architecture-conformance test and UML-export validation), 25 Chromium browser tests (roles, use cases, accessibility of all three role pages), and a GitHub Actions workflow that runs them on every pull request to `main`.

The two primary end-to-end use cases demonstrated are UC2 *Run a trip* with UC1 *Track shuttles* (driver → server → many students), and UC3 *Waiting request* feeding UC4/UC5 (student → office demand view → forecast). Both are covered by automated integration and Chromium browser tests and were walked through in a browser on 31 October – 1 November 2026.

8.2 What was not built

Single sign-on and integration with university systems; native apps and push notifications; HTTPS configuration (development runs on `localhost`); field test with a real vehicle and phone GPS; timetable data (no schedule fallback when GPS is lost — the system shows "last seen" instead); multi-instance deployment; Arabic localisation; formal accessibility audit (automated axe WCAG checks and keyboard tests pass in Chromium for the student, driver and office pages, but no screen-reader session or physical-device check was done); a driver client that keeps reporting in the background (§6.5); tests in browsers other than Chromium; a trained ML model (deliberately, §6.7).

8.3 How the prototype maps onto the architecture

Component (Figure A.11)	Folder	Key classes	Tests
Web API	`src/web/`	`createApp`, route modules, `middleware.ts`	`test/integration/api.test.ts`
Real-time Gateway	`src/realtime/`	`SseHub`, `LiveView`	`test/integration/realtime.test.ts`
Identity & Access	`src/identity/`	`AuthService`, `TokenService`, password hasher	`test/unit/security-and-events.test.ts`
Network Management	`src/network/`	`NetworkService`, `RoutePath`	`test/unit/route-path.test.ts`
Tracking	`src/tracking/`	`TrackingService`, `TripTracker`, `SignalState`, ETA strategies	`test/unit/tracking-service.test.ts`, `trip-tracker`, `signal-state`, `eta`
Demand & Forecasting	`src/demand/`	`WaitingService`, `DemandService`, predictors, `backtest`	`test/unit/prediction.test.ts`, API tests
Event Bus	`src/shared/`	`EventBus`	unit tests
Persistence	`src/persistence/`	SQLite repositories, in-memory repositories, migrations	API and recovery tests

Component (Figure A.11)	Folder	Key classes	Tests
Composition root	`src/container.ts`	`buildContainer`	all integration tests
Browser clients	`public/`	`student.js`, `driver.js`, `office.js`, `gps.js`, `map.js`, `live-feed.js`	`e2e/*.spec.ts` (Chromium), `test/unit/simulated-gps.test.ts`, `driver-trip-ending.test.ts`

8.4 Technology choice

We chose **TypeScript on Node.js 24 LTS with Express 5 and SQLite (Node's built-in `node:sqlite`)**, and plain HTML/CSS/JavaScript modules in the browser. Node's event loop handles thousands of long-lived SSE connections cheaply, which is exactly the real-time workload; TypeScript makes the component interfaces (ports) explicit and checked; Node 24 runs TypeScript directly, so there is no build step to break; `node:sqlite` gives a transactional database with zero installation; and the whole stack is free, widely documented and runs offline on any student laptop. The runtime has a single third-party dependency (Express). The trade-offs: `node:sqlite` is still marked experimental in Node 24 (we pin `engines` and test on 24.14), and without a front-end framework the UI code is more manual.

8.5 Evidence

Check	Result (31 Oct – 7 Nov 2026, Windows 11, Node 24.14.1)
`npm run typecheck`	passes, no errors
`npm test`	119 tests, 119 passed, 0 failed — `docs/evidence/test-run.txt`
`npm run test:browser`	25 Chromium tests, 25 passed (Playwright 1.64.1, Chrome for Testing 156) — `docs/evidence/browser-test-run.txt`. The three office-page tests added on 6 November failed before their fix (`docs/evidence/accessibility-office-2026-11-06.txt`)
`npm audit`	0 vulnerabilities in all dependencies, including development tools — `docs/evidence/npm-audit.txt`
GitHub Actions (`verify`)	required for every merge into `main`; passed on `main` after each merge, most recently run 41514123139 after PR #16 (2 Nov)
`npm run load-test` (1,000 viewers; local benchmark)	31 Oct: p50 23 ms, p95 35 ms, p99 47 ms, max 56 ms. 1 Nov: p50 25 ms, p95 37 ms, p99 48 ms, max 55 ms; 60,000/60,000 delivered; 0 skipped — `docs/evidence/load-test-1000*.txt`
`npm run load-test -- --viewers 3000 --rounds 10`	31 Oct: p95 90 ms, max 108 ms. 1 Nov: p95 98 ms, max 113 ms; 90,000/90,000 delivered; 0 skipped — `docs/evidence/load-test-3000*.txt`. Same process, loopback, one shared session, no browsers
`npm run evaluate`	see §6.7 — `docs/evidence/demand-evaluation.txt`
Browser walkthrough (student, driver, office)	all three interfaces exercised against a running server with the fleet simulator (`docs/test-plan.md` §5), and automated in Chromium since PR #13. On 6 November an automated browser session observed UC1's full loop lap and UC4's historical trace, and ran the demonstration sequence with timings (`docs/evidence/walkthrough-2026-11-06.txt`). These are tool-driven sessions, not a walkthrough by either member

Check	Result (31 Oct – 7 Nov 2026, Windows 11, Node 24.14.1)
`npm run uml` / `npm run uml:check`	7 Nov: diagram 10 re-rendered after two of its interface labels were found printed over each other; the other 15 re-rendered byte-identical. 2 Nov: all 16 diagrams rendered with the checksum-verified PlantUML MIT 1.2026.9 (Java 21, Smetana) and validated: SVGs parsed as well-formed XML, PNGs fully decoded, none clipped, all from the current sources. Zuhair's review of PR #15 showed that the first validator accepted malformed content; it now rejects it. Zuhair's independent render of the same sources (Java 17, Linux) matched in structure. Appendix A prints every diagram at 62 % or more of its rendered size, with overview and detail pages for wide ones; there the smallest annotations print at 5.5 pt. The separate UML-set PDF ("full readable size") now enlarges each diagram towards a 7 pt minimum without cutting class boxes across rows of pages: 9 of the 16 diagrams reach 6.9 pt or more (5 before), and the smallest text is A.9's stereotype labels at 5.7 pt, limited by the A4 page (sizes measured from the build, `docs/evidence/uml-inspection-2026-11-06.txt`). The team's own visual check, including on paper, is still open (issue #1)

9. Reflection

> **Draft retrospective — personal confirmation OUTSTANDING.** The brief asks what we would do differently, and rewards honesty. This draft was prepared from the project record (the decisions, defects and reviews in `docs/test-plan.md` §4 and the pull requests) with the tool declared in §10.2. It states no personal feelings or opinions. Each member must read it, correct what does not match their own experience and add their own reflection before submission.

1. Spread the work over the semester. The repository record starts on 31 October (Week 9). Almost all recorded design, build and review work happened between 31 October and 2 November, with a completion pass on 6–7 November. The record does not show the Week 4 proposal approval or the Week 9 checkpoint (§3.3). Starting the repository at the proposal, with a dated log, would have left time for stakeholder input and a field test. It would also have produced the timeline the brief asks for.

2. Talk to real stakeholders before fixing the numbers. No interviews or observations are recorded (§2.4). The 1-second freshness target, the 20 s / 60 s signal thresholds and the 20-minute waiting expiry are reasoned, not elicited. They drive the architecture (§6.1), and §6.3 shows that the push-versus-polling decision depends on the first of them alone. A short interview with transport-office staff and a few students would be the first thing to do differently.

3. Check content, not existence. The first UML render script only checked that files existed; Zuhair's first render found empty and clipped images (test-plan item 14). The replacement validator then accepted malformed SVG and PNG content, which his review of PR #15 reproduced (item 16). Each check encoded its author's assumptions; an independent reviewer with a different method found what the author's tests could not.

4. Test where the defects live, on every page. Browser tests found UI defects that unit tests could not see (items 10, 11, 15 and the PR #13 fixes). The office page had no automated accessibility check until 6 November; when it got one, it failed three WCAG criteria at once (item 17). The same checks should have covered all three role pages from the start.

5. Keep claims at the strength of the evidence. Several statements were weakened after review: QA-4's "no position lost" became a bounded guarantee; the load test is now described as a same-process benchmark; the draft architecture scores let the polling transport lower option C's modularity score and

cited a brief requirement that does not exist (§6.3). Separating what was measured, what is arithmetic and what is judgement, in every claim, would have avoided these corrections.

6. Simulation is not reality. All GPS behaviour was exercised with the simulator, and the thresholds are untested on a phone. The 6 November walkthrough added a limit the simulator alone had hidden: a driver page in a background tab stops reporting (§6.5). A field test on a real vehicle would come first, and the driver client may need to be a native app.

7. No data, no model. Not training a model on invented data was the most important forecasting decision (§6.7). The synthetic backtest proves only that the pipeline works.

8. Ownership and tool use. The baseline and most later corrections were produced with the AI tools declared in §10.2, from the members' accounts and at their request. The members' recorded contributions are the reviews, verifications and corrections listed in §10.1. The course allows AI use with disclosure, but not submitting AI-generated analysis as one's own thinking, and the instructor may ask either member to defend any diagram. A process in which each member first derives part of the design themselves, such as the §6.3 scores or the analysis models of the Week 6 studio, would make the contribution statement and the viva easier to support.

9. Repository process. Protected `main` and reviewed pull requests caught real defects before merge (items 14–16). One process lesson: a GitHub rebase merge on 2 November recorded a personal e-mail address as committer on five public commits; later merges used merge commits after the privacy setting was confirmed. Account settings should be checked before the first merge on a public repository.

Other engineering observations: the load test found a corner-snapping defect (item 1); an integration test found work running after shutdown (item 2); an independent review on 1 November found a trip ended locally but not on the server, and live streams outliving their session (items 5–6).

10. Contribution statement and AI-use statement

10.1 Contribution statement

Factual record. From the Git history, the pull requests and their reviews. It records which account submitted or approved what. It does not say who personally wrote, read or checked each line; the tools used are in §10.2. The detailed log is in `docs/team/work-ownership-plan.md`, with Zuhair's record in `docs/team/zuhair-contributions.md`.

	Abdullah (`abdullahshammari796-ctrl`)	**Zuhair Halawa (`zuhairhalawa`)**
Baseline	Repository created and the baseline imported from this account on 1 Nov (`8d11ad6`). The baseline was generated with Claude Code (§10.2) and is not either member's own authored work	—
Pull requests submitted	#14 (GPS-source resumption, waiting-refresh fix, dependency fix, UML source corrections); #15 (validated UML rendering, readable diagram pages, sign-in form fix, corrections after review); #16 (records and wording); the final completion pull request (6–7 Nov)	#12 (same-weekday median forecaster, issue #6); #13 (browser and simulator tests, accessibility and mobile fixes, waiting-state fixes, use-case evidence, ADR evidence review, CI workflow; issues #5, #7, #8, #9)

	Abdullah (`abdullahshammari796-ctrl`)	**Zuhair Halawa (`zuhairhalawa`)**
Commits, excluding merges (to 2 Nov)	14, including the baseline import and the set-up commit	8
Reviews posted	Approvals of #12 and #13 (prepared with Claude Code; they record the checks that were run, not a personal line-by-line reading)	Approval of #14; changes requested on #15 (validator defect, A.3 layout) and approval of its corrections; approval of #16
Repository administration	Branch protection, issues, merges of #12–#16	—
Independent verification	—	First render of the 16 UML sources (empty and clipped exports found); Linux reproductions with an independent decoder; visual inspection of PDFs and slides

Personal statements and signatures — OUTSTANDING. Each member lists only work they actually did, in their own words: design, models, code, tests, documentation and presentation. Student IDs and signatures are added only in the private submission copy, not in this public repository.

Member	**What I personally did**	**Signature**	**Date**
Abdullah	*OUTSTANDING*	*(private copy)*	
Zuhair Halawa	*OUTSTANDING*	*(private copy)*	

10.2 AI-use statement

The brief asks which AI tools were used, for what, and what was changed. The full record is in `docs/team/ai-use-statement.md`.

- **Tools.** Claude Code (Anthropic), used from Abdullah's computer and GitHub account. ChatGPT (OpenAI), including its Work mode, connected to Zuhair's GitHub account. Any other tool a member used is **OUTSTANDING**.
- **When.** 31 October – 7 November 2026.
- **Which parts, and for what.**

Part	**Claude Code (Abdullah's account)**	**ChatGPT (Zuhair's account)**
Requirements, architecture and §6 analysis	Drafted §1–§3 and §6, the ADRs and the QA scenarios; produced the §6.3 technical assessment (6 Nov)	Evidence review of ADR-0001, 0002, 0003 and 0008 (PR #13)
UML	Wrote and corrected the 16 PlantUML sources; rendered and validated them; built the PDF layout	First independent render; found export defects
Prototype code and tests	Generated the baseline code and tests; fixed the defects numbered 1–18 in `docs/test-plan.md` §4	PR #12 forecaster and tests; PR #13 browser tests and UI fixes
Documents and presentation	Drafted the proposal, this report (including the §9 draft and the §10.1 factual record), the test plan, the deck, the speaking script and the demo runbook	Use-case evidence matrix; review findings on PDFs and slides

Part	Claude Code (Abdullah's account)	ChatGPT (Zuhair's account)
Verification and repository	Ran checks, load tests and browser walkthroughs; created the repository; opened issues and pull requests; merged after approval	Reviewed PRs #14–#16 on a fresh Linux checkout and posted the reviews

- **What changed after the baseline.** Eighteen defects and corrections are recorded in `docs/test-plan.md` §4. Among them: two code defects and two overclaims found by an independent review on 1 November; UI defects found by Zuhair's browser tests (PR #13); two defects found in the owner review of PRs #12/#13; the export-validation defects found by Zuhair's render and his review of PR #15; and the office-page accessibility failures and UML print sizes found on 6 November. The draft §6.3 scores were also corrected (§6.3).
- **What the tools did not do.** They did not register the team, choose the topic, contact the instructor or submit anything for marking. They did not invent rubric details, signatures, student IDs, dates, contributions, field observations or real-world data; missing facts are marked **OUTSTANDING** and demonstration data is labelled synthetic.
- **What each member personally changed or verified: OUTSTANDING.** For example: which parts each member read, checked, decided or rewrote, and which decisions they disagree with.

Appendix A (UML diagram set) follows. Appendix B — requirements traceability: `docs/00-requirements-traceability.md`. Appendix C — test plan and results: `docs/test-plan.md`.

Appendix A — UML diagram set

Campus Shuttle Tracking & Demand Prediction · BCS 305 Software Architecture · Fall 2026

All diagrams are drawn with **PlantUML** (a text-based UML tool) using standard UML 2 notation. The editable sources are in `docs/uml/*.puml`; rendered SVG/PNG files go to `docs/uml/out/` (`npm run uml`). The class, component and sequence diagrams were written from the actual source code and were compared with it member by member on 1 November 2026 (corrections are listed in `docs/test-plan.md` §4). They were compared again in the owner review after PRs #12 and #13, which corrected diagrams 02, 04, 05, 07–12, 14 and 16 (`docs/test-plan.md` §4, item 13). **Rendering (2 November 2026; diagram 10 again on 7 November after a label fix):** all 16 sources render with the checksum-verified PlantUML MIT 1.2026.9 (built-in Smetana layout, no Graphviz). `npm run uml` validates every export (complete, unclipped, rendered from the current source) and records it in `docs/uml/out/manifest.json`; `npm run uml:check` repeats that check without Java. Every SVG is checked as well-formed XML and every PNG is fully decoded. No diagram is printed below 60 % of its rendered size: one that is too large for a page gets a vector overview page (zoom in on screen) followed by overlapping detail pages. **Two printings.** In the report (Appendix A) the reading pages print most labels at 6.2–9.8 pt and the smallest annotations at 5.5 pt. The separate UML-set PDF, which the brief asks for at full readable size, enlarges each diagram towards a 7 pt minimum without cutting class boxes across more rows of pages: 9 of the 16 diagrams then print their smallest text at 6.9 pt or more (5 before), and the smallest is 5.7 pt (stereotype labels in A.9, whose 1,502-pixel height limits it on an A4 page; print A.9 and A.10 on A3 or read them on screen). These sizes were measured from the build, not from paper; nobody has yet checked a printed copy (issue #1). Only one aspect is checked automatically: `test/architecture.test.ts` fails if a source file imports a component that the component diagram does not allow. It does **not** check UML operations, attributes, associations, multiplicities or sequence messages; those were reviewed by hand and can drift when the code changes. Where a diagram writes a type as `{…}`, the code uses an inline TypeScript object type with no class name.

#	Diagram	Required by the brief	Source file
A.1	Use case diagram	Yes	`01-use-case-diagram.puml`
A.2	Use case descriptions (5 primary use cases)	Yes (≥ 3)	this document
A.3	Activity diagram — Run a trip	Yes	`02-activity-run-trip.puml`
A.4	System sequence diagram — Run a trip and track it	Yes	`03-system-sequence-diagram.puml`
A.5	Design sequence diagram — processing a location upload	Extra	`04-sequence-location-update.puml`
A.6	State machine — Trip and GPS signal	"Where relevant"	`05-state-machine-trip.puml`
A.7	State machine — Waiting request	"Where relevant"	`06-state-machine-waiting-request.puml`
A.8	Domain model (conceptual class diagram)	Yes	`07-domain-model.puml`
A.9	Design class diagram 1/2 — tracking, network, real-time	Yes	`08-class-diagram-tracking.puml`
A.10	Design class diagram 2/2 — demand, forecasting, identity	Yes	`09-class-diagram-demand-identity.puml`
A.11	Component diagram	Yes	`10-component-diagram.puml`

#	Diagram	Required by the brief	Source file
A.12	Deployment diagram	Yes	`11-deployment-diagram.puml`
A.13–A.17	Design pattern class diagrams (Strategy, Observer, State, Repository, Adapter)	Yes (≥ 3 patterns)	`12`–`16-pattern-*.puml`

A.1 Use case diagram

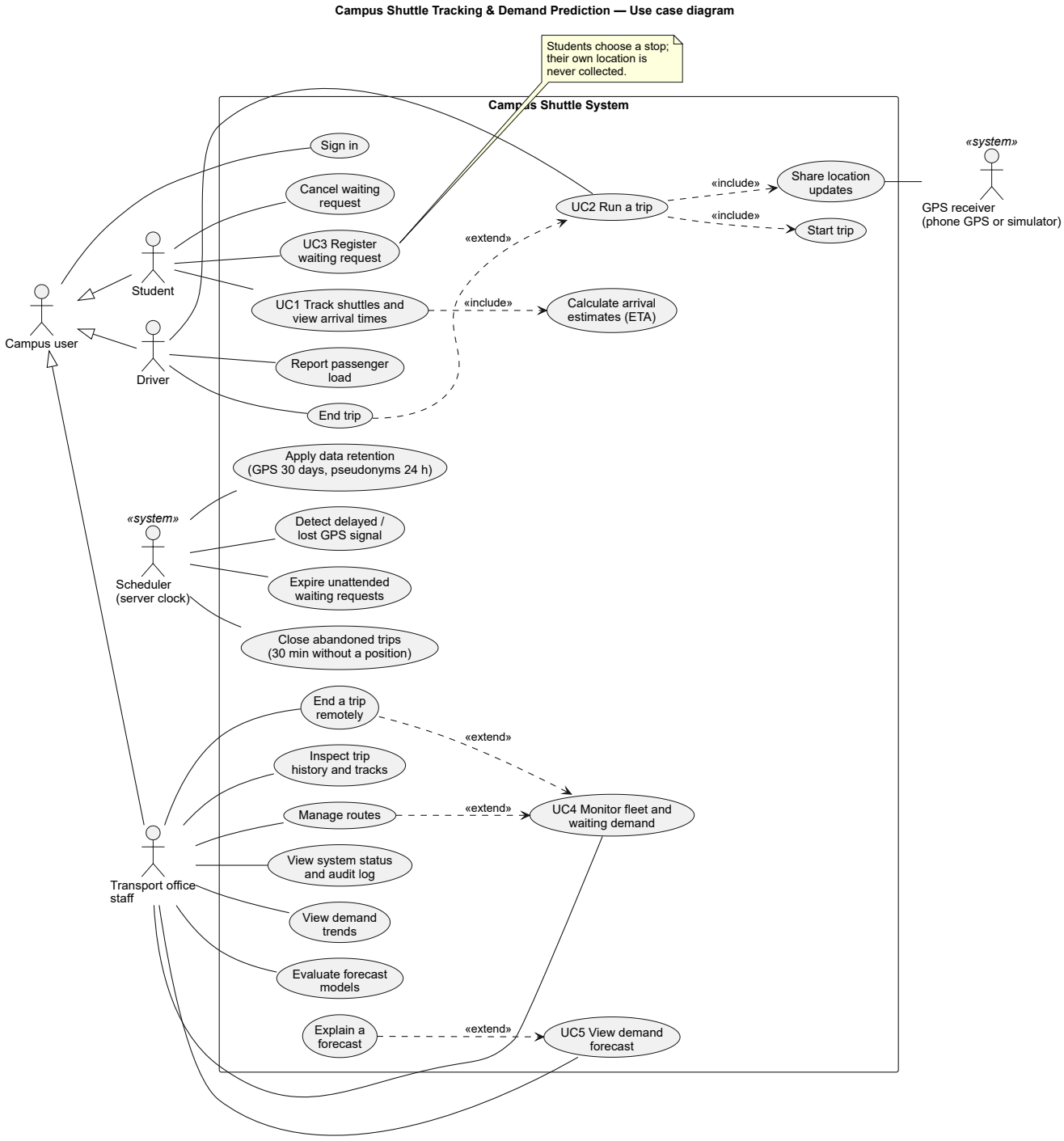


Figure A.1 — Use case diagram — overview at 55 % (read the 2 detail pages that follow, at 83 %; on screen, zoom in)

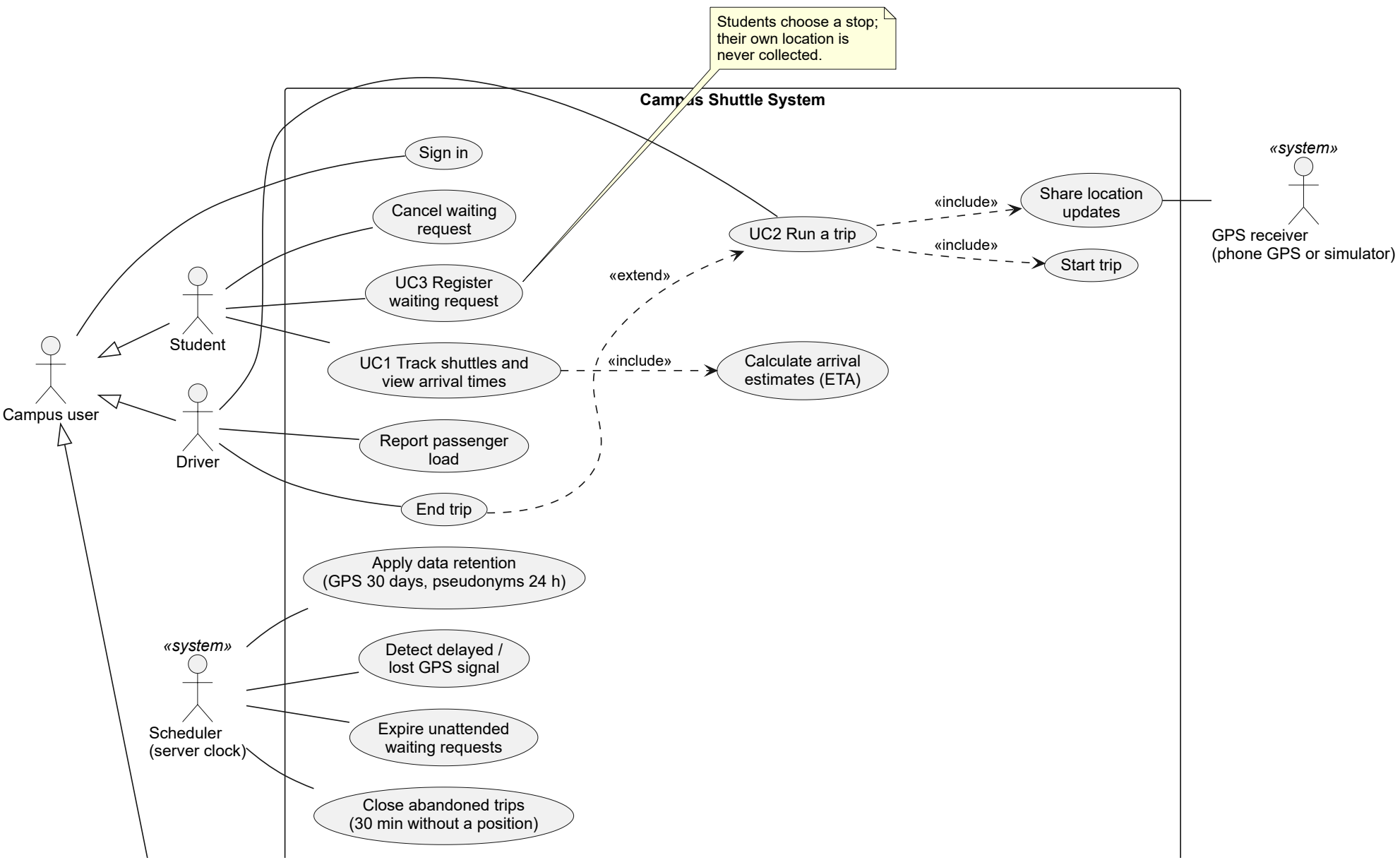


Figure A.1 — Use case diagram — detail 1 of 2 (row 1 of 2), at 83 %

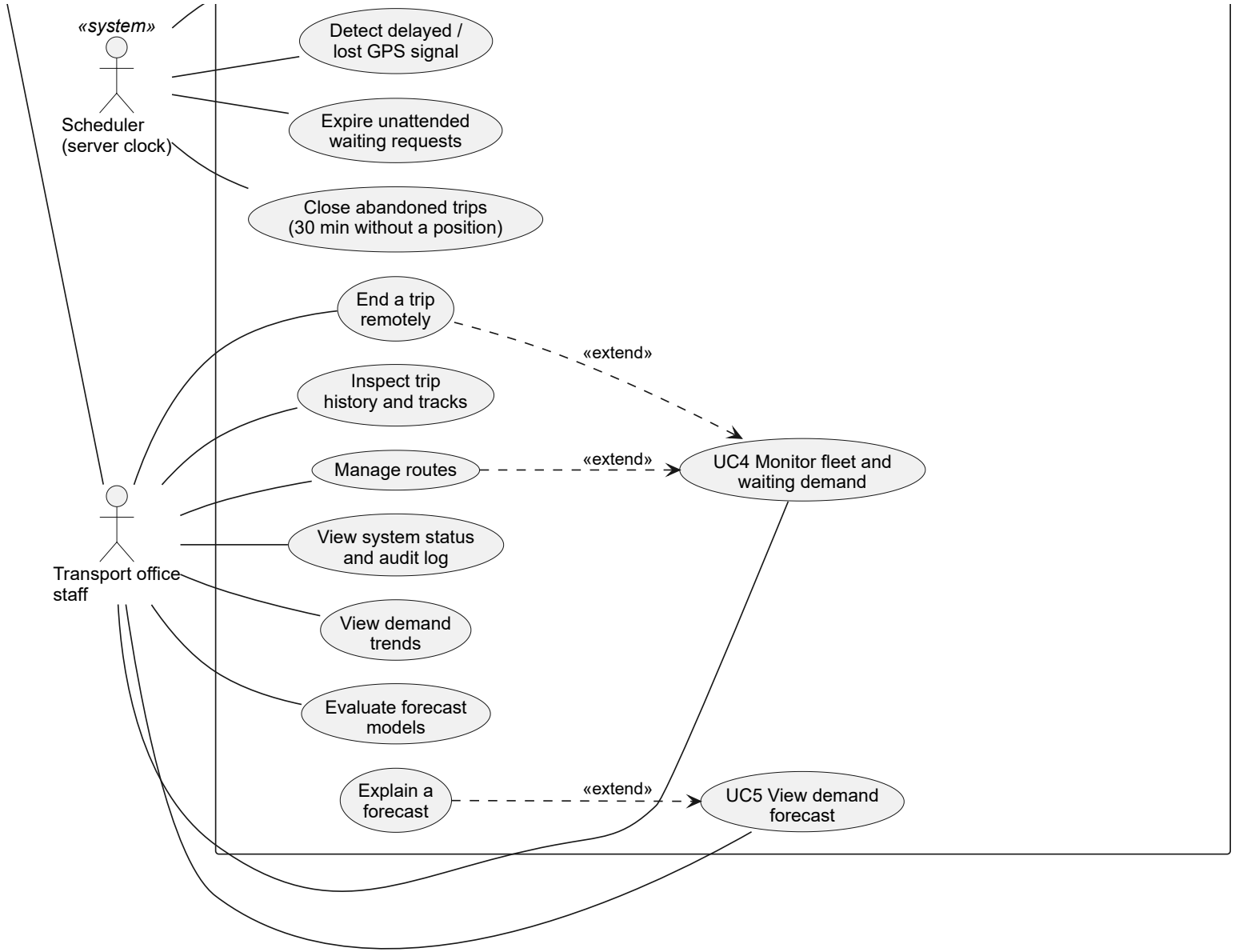


Figure A.1 — Use case diagram — detail 2 of 2 (row 2 of 2), at 83 %

Three primary human actors (Student, Driver, Transport office staff) share a generalised *Campus user* actor for sign-in. Two system actors appear as secondary actors: the **GPS receiver** (the phone's GPS or the built-in simulator) and the **Scheduler** (the server clock that detects stale GPS and expires waiting requests). `«include»` marks behaviour that always happens as part of a use case; `«extend»` marks optional behaviour.

A.2 Use case descriptions

The five primary use cases below are all implemented end to end in the prototype. UC1 and UC2 together form the main demonstration scenario.

UC1 — Track shuttles and view arrival times

Field	Description
Primary actor	Student
Stakeholders and interests	Student: wants to know whether to walk to the stop now, without waiting in the heat. Transport office: fewer phone calls asking "where is the bus?". Driver: wants passengers ready at the stop.
Preconditions	Student is signed in. At least one route is in service.
Trigger	Student opens the student page.
Main success scenario	1. System shows the schematic map with routes, stops and every active shuttle, and opens a live feed. 2. Student selects a stop (on the map or from a list; the choice is remembered on that device). 3. System lists every shuttle on a route serving that stop, soonest first, with the estimated minutes, clock time, distance, passenger load and a live-signal badge. 4. As drivers send positions, the system pushes updates; positions and minutes change without reloading. 5. When a shuttle on a loop route reaches the selected stop, its next-lap estimate is recalculated and the list is sorted again. A non-loop route does not invent a next lap.
Extensions	3a. No shuttle is running on a route serving the stop: system says so plainly. 4a. A shuttle's newest position is more than 20 s old: its estimate is shown as approximate ("~6 min") with "location delayed"; when the estimate time has passed it shows "Due". 4b. No position for more than 60 s: the system shows "GPS signal lost · last seen N min ago" and **no estimate**. 4c. The student's connection drops: a banner says "reconnecting — times may be out of date"; on reconnection a full snapshot replaces the old state. 4d. Session expired: banner offers to sign in again.
Postconditions	None (read-only use case).
Quality attributes	Freshness (QA-1), scalability with many viewers (QA-2), honesty under GPS loss (QA-3), privacy: no driver identity is ever sent to students (QA-6).
Implemented in	`public/js/student.js`, `src/web/routes/shared-routes.ts`, `src/realtime/sse-hub.ts`, `src/tracking/tracking-service.ts`

UC2 — Run a trip (start, share location, end)

Field	Description
Primary actor	Driver. Secondary actor: GPS receiver (device GPS or simulator).
Stakeholders and interests	Driver: minimal interaction while working. Students: accurate positions. Office: knows which vehicle is on which route and who is accountable.
Preconditions	Driver is signed in, has no other active trip, and the chosen shuttle is free.
Trigger	Driver taps "Start trip" after choosing route, shuttle and location source.

Field	Description
Main success scenario	1. System validates the route is in service and that the shuttle and driver are free, then creates an ACTIVE trip (signal "awaiting first GPS position") and announces it to all viewers. 2. Driver app starts the GPS source. 3. The simulator obtains a position every ~3 s (device GPS is event-driven and throttled to at most one fix per 2 s) and uploads all queued positions in one batch with the device's current time. 4. System corrects device clock skew, validates and orders the positions, stores them, snaps the shuttle onto the route, updates its smoothed speed, recalculates ETAs, detects stops reached, and pushes one update to all viewers. 5. App shows "what passengers see" (Live / Delayed / GPS lost) and the next stop. 6. Driver optionally sets passenger load (seats available / filling up / full). 7. Driver taps "End trip" and confirms; the app first delivers the queued positions (waiting at most 5 s), holds new ones and sends the end request; the system completes the trip and removes the shuttle from every map; only after the server confirms does the app stop GPS and show "Trip ended".
Extensions	1a. Shuttle or driver already on an active trip, or route out of service: system refuses with the reason (HTTP 409; an unknown route or shuttle gives 404). 3a. GPS produces no positions (tunnel, outage): nothing is uploaded; students see the delayed → lost sequence (UC1 4a–4b). 3b. Network unavailable: positions stay queued in the open driver page (newest 500 kept; older ones dropped and counted; lost if the page is closed or reloaded) and are retried with exponential backoff (1 s → 15 s); when the network returns they are delivered in batches of up to 50 and the shuttle jumps to its true position; positions more than 10 minutes old on arrival are rejected. 4a. A position is duplicated, out of order, physically impossible (> 110 km/h jump), too inaccurate (> 150 m), too old (> 10 min) or outside the service area: it is rejected with a reason code and the rest of the batch is still accepted. 4b. The page is reloaded: the trip resumes from the server's last known position with the same location source as its latest accepted position (device GPS or simulator). If no position has been accepted yet, the app shares nothing until the driver chooses a source. 7a. The office ended the trip remotely (UC4): the next upload returns 409/404 and the app returns to the start screen with an explanation. 7b. No accepted position for 30 minutes (counted from the last position, or from the trip start if there was none): the system closes the trip automatically (TERMINATED, reason NO_SIGNAL_TIMEOUT). 7c. The end request fails (no network, expired session, server unreachable): the trip stays ACTIVE, location sharing continues, the app says the trip was **not** ended and the driver can try again; if the server answers that the trip had already ended (409/404), the app treats it as ended.
Postconditions	The trip is COMPLETED or TERMINATED, with its GPS trace stored (deleted after 30 days).
Implemented in	`public/js/driver.js`, `public/js/gps.js`, `src/web/routes/driver-routes.ts`, `src/tracking/*`

UC3 — Register a waiting request ("I'm waiting here")

Field	Description
Primary actor	Student
Stakeholders and interests	Student: wants the shuttle to know someone is waiting. Office: real-time demand signal and history for forecasts. Privacy officer: no tracking of individual students.
Preconditions	Student is signed in and has selected a stop.
Main success scenario	1. Student taps "I'm waiting here", optionally choosing a specific route. 2. System records the request against a **pseudonymous reference** (keyed hash of the user id), sets a 20-minute expiry, and pushes the stop's new waiting count to all viewers. 3. The student sees "You're on the waiting list at Library · expires in 20 min". 4. When a shuttle on a matching route reaches the stop, the system marks the request SERVED and the student sees "A shuttle reached Library at 08:14".

Field	Description
Extensions	1a. Same stop and route requested again: the existing request is returned (idempotent). 1b. Different stop or route: the previous request is cancelled and replaced. 1c. Route does not serve the stop: rejected with a reason. 1d. More than 6 requests in an hour: rejected (429) to prevent abuse that would distort demand data. 3a. Student cancels: request CANCELLED and the count updates. The open page reconciles its private request after a matching waiting-count event and after a reconnect snapshot, including closures missed while offline. 4a. No shuttle within 20 minutes: request EXPIRED.
Postconditions	Request closed in one of SERVED / CANCELLED / EXPIRED; 24 h later its pseudonymous link is erased and it remains only as anonymous demand data.
Implemented in	`src/demand/waiting-service.ts`, `src/web/routes/student-routes.ts`, `public/js/student.js`

UC4 — Monitor the fleet and waiting demand (with remote trip termination)

Field	Description
Primary actor	Transport office staff
Preconditions	Staff member signed in with the office role.
Main success scenario	1. System shows KPI tiles (shuttles on trip, GPS problems, students waiting, live viewers), the fleet map with waiting badges, waiting counts per stop, and a table of active trips including the accountable driver, GPS state, last position age, load and next-stop ETA — all updated live. 2. Staff opens a trip's track to inspect its GPS trace: positions, distance, duration, longest gap and number of gaps longer than the stale threshold.
Extensions	1a. *End a trip remotely* («extend»): staff chooses "End", must give a reason; the system terminates the trip, records actor, time and reason in the audit log, and the driver app is told on its next contact. 1b. *Manage routes*: take a route out of service or back, edit name/colour/planned speed/dwell time, or create a new route from an ordered list of stops (created inactive for review). Every change is audited.
Implemented in	`public/js/office.js`, `src/web/routes/office-routes.ts`, `src/network/network-service.ts`

UC5 — View the demand forecast (with explanation)

Field	Description
Primary actor	Transport office staff
Main success scenario	1. Staff opens "Demand & forecast", picks a date and a model. 2. System shows the expected requests per hour (all stops), a stop × hour heat map, the peak hour and busiest stop, and states which data sources were used (synthetic and/or live). 3. Staff selects a cell; the system explains it: method, the exact past observations used (dates, counts, weights) and the plausible range. 4. Staff compares models in the backtest table (MAE, RMSE, bias, range coverage).
Extensions	2a. Not enough history for a stop-hour: the cell shows "–" and **no number is invented**. 2b. History contains synthetic data: a warning banner states that the figures illustrate the method only.
Implemented in	`src/demand/demand-service.ts`, `src/demand/prediction/*`, `public/js/office.js`

A.3 Activity diagram — Run a trip

Swimlanes separate the driver app from the server. The two interruption paths are visible: *GPS outage* (no position is appended to the queue) and *network loss* (the queue is kept and retried with backoff). Two further paths are shown: the server ending the trip while it runs (office or 30-minute auto-close), and ending the trip, where the outer loop repeats until the server confirms the end, so a failed request leaves the trip active.

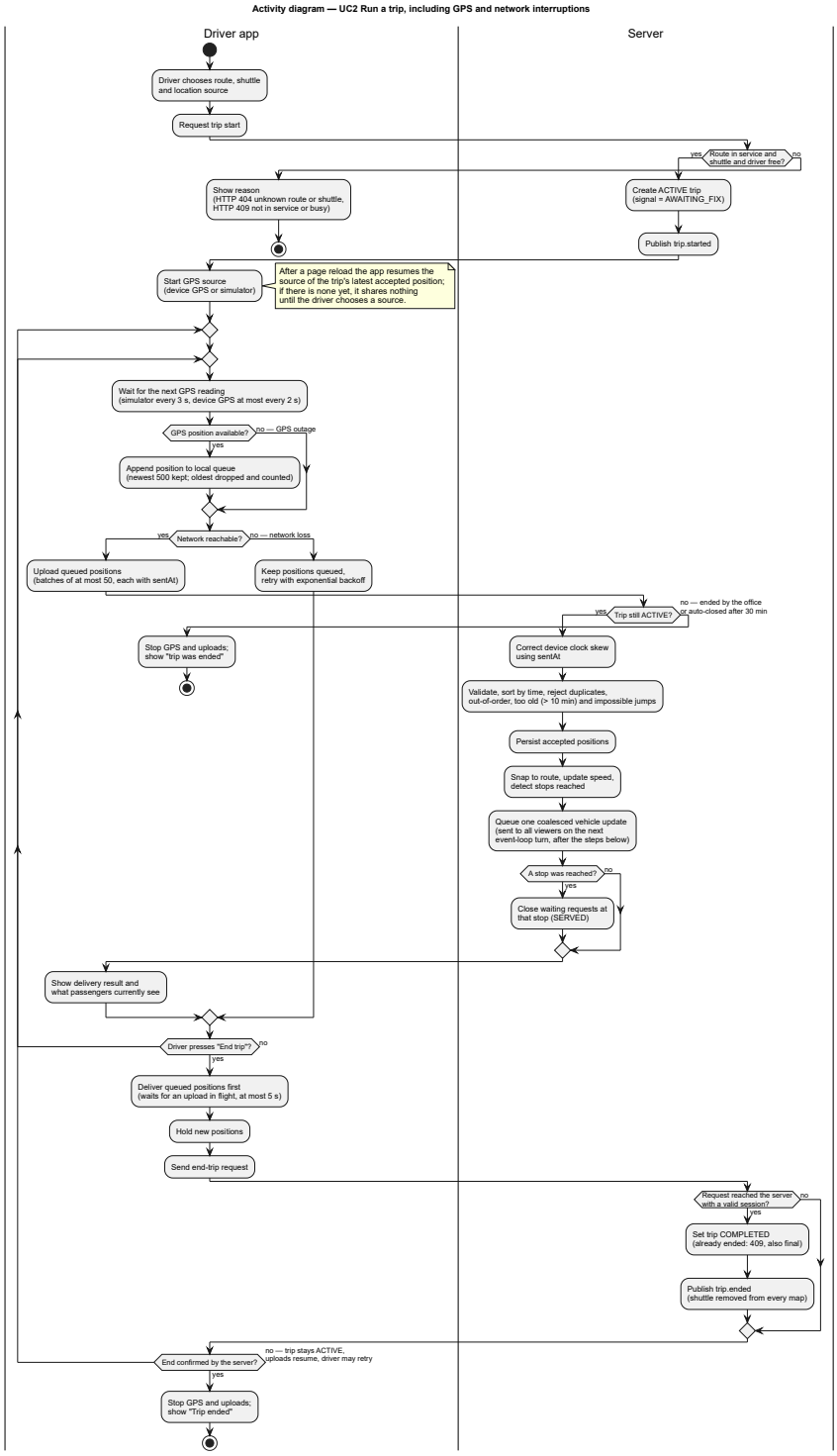


Figure A.3 — Activity diagram: Run a trip with GPS and network interruptions — overview at 34 % (read the 3 detail pages that follow, at 75 %; on screen, zoom in)

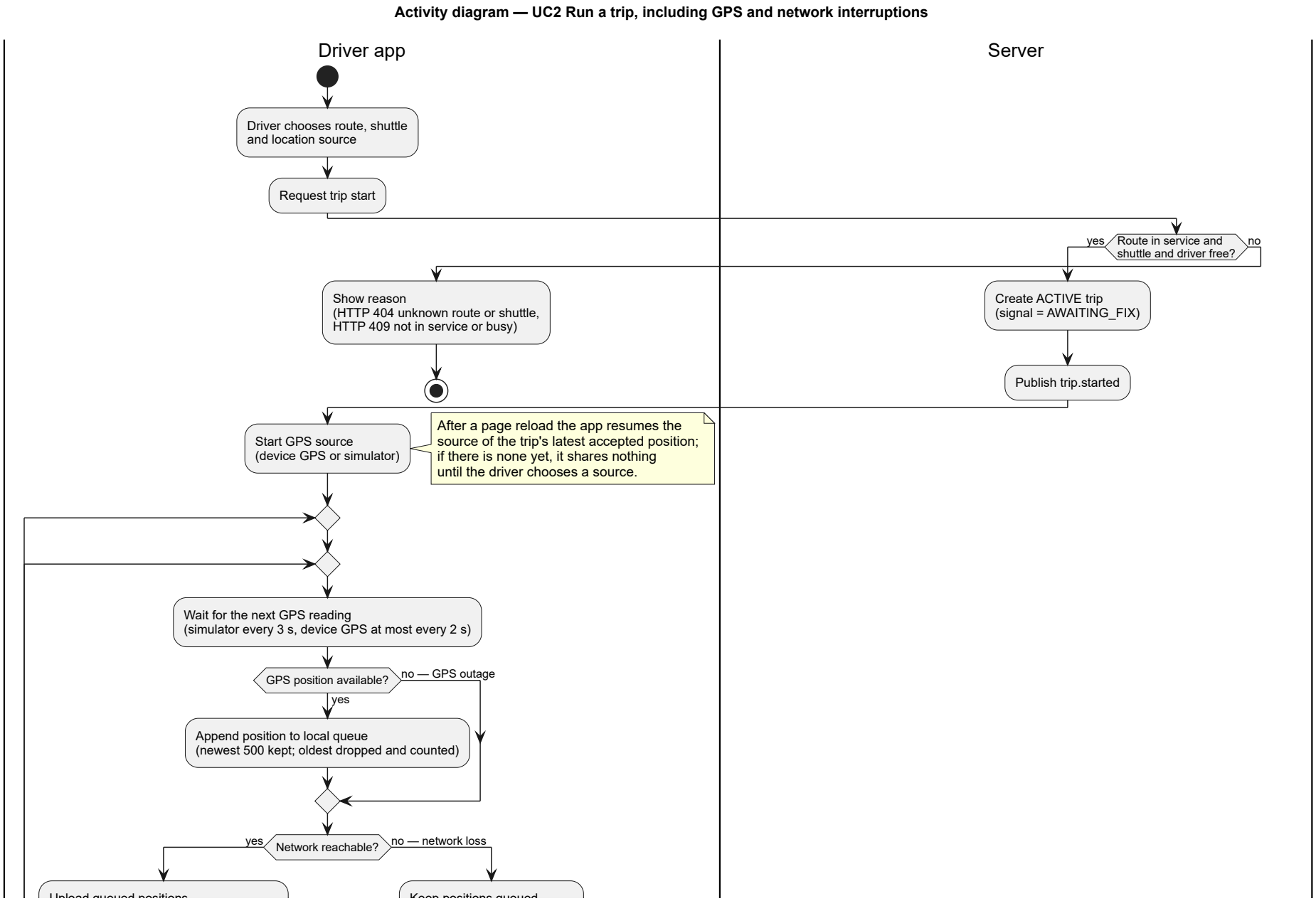


Figure A.3 — Activity diagram: Run a trip with GPS and network interruptions — detail 1 of 3 (row 1 of 3), at 75 %

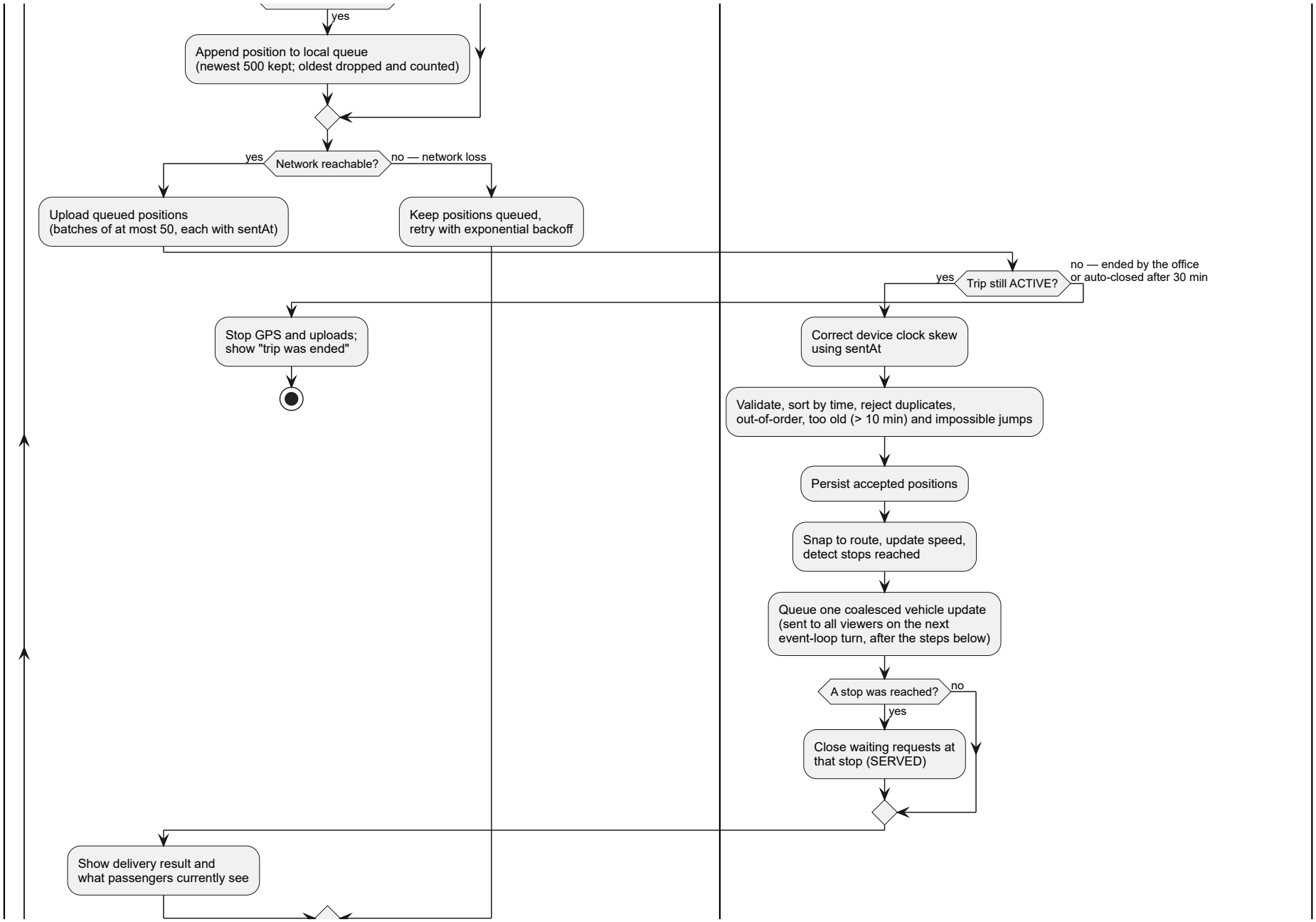


Figure A.3 — Activity diagram: Run a trip with GPS and network interruptions — detail 2 of 3 (row 2 of 3), at 75 %

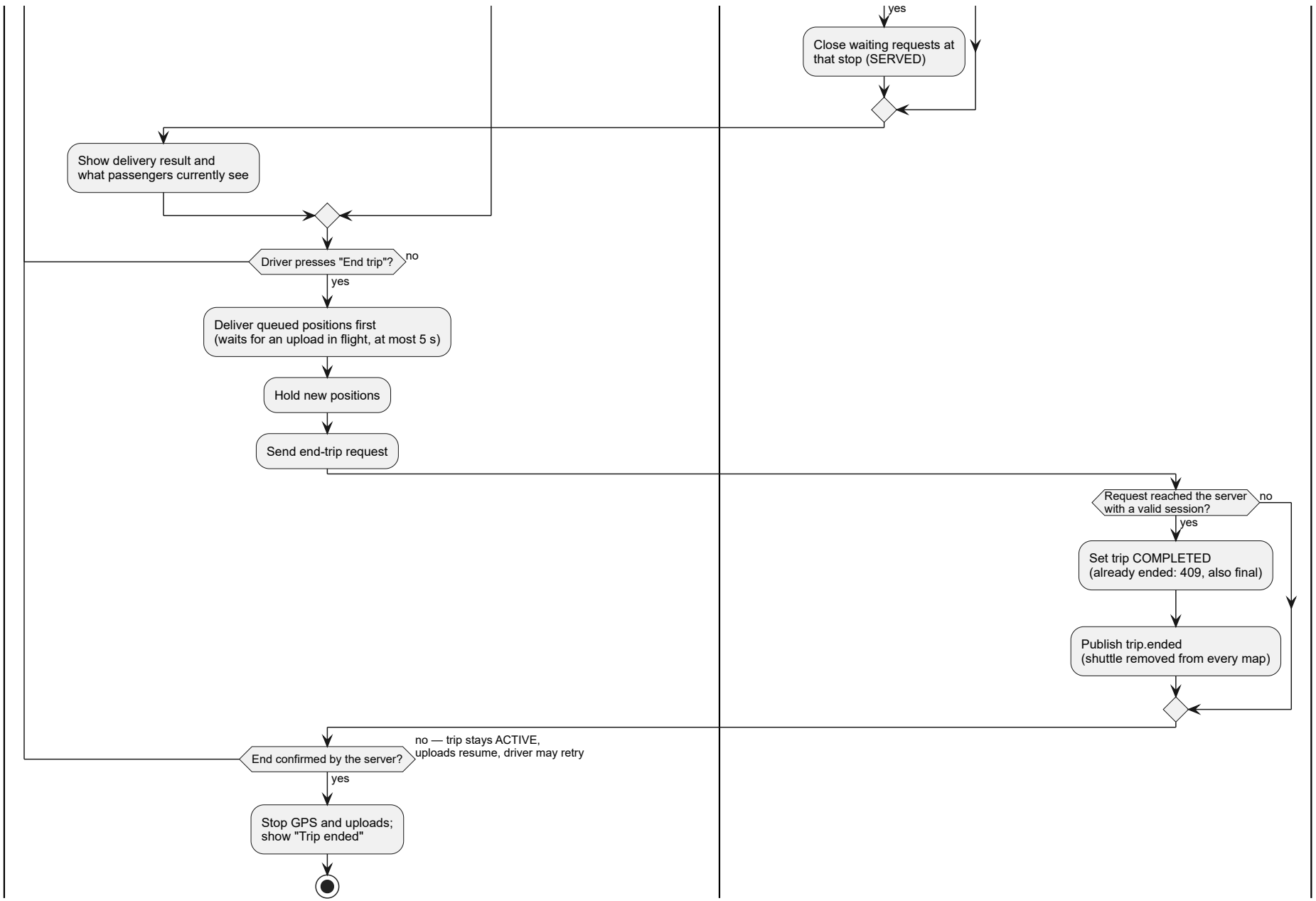


Figure A.3 — Activity diagram: Run a trip with GPS and network interruptions — detail 3 of 3 (row 3 of 3), at 75 %

A.4 System sequence diagram — Run a trip and track it

The system is a black box; messages are the system operations of the REST API and the live feed. The `opt` fragments are system-initiated (time) events.

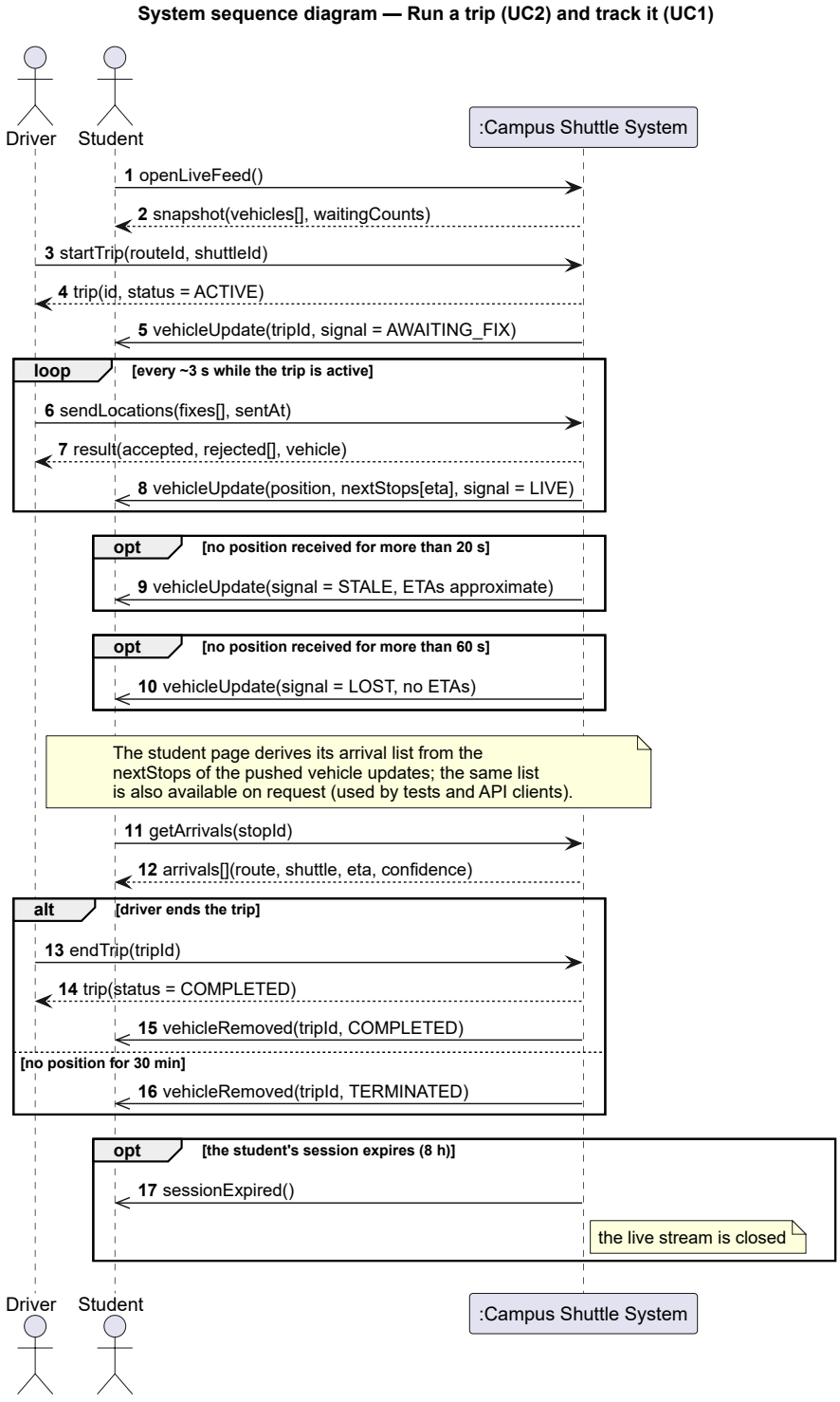


Figure A.4 — System sequence diagram for UC2 + UC1

A.5 Design sequence diagram — processing one location upload

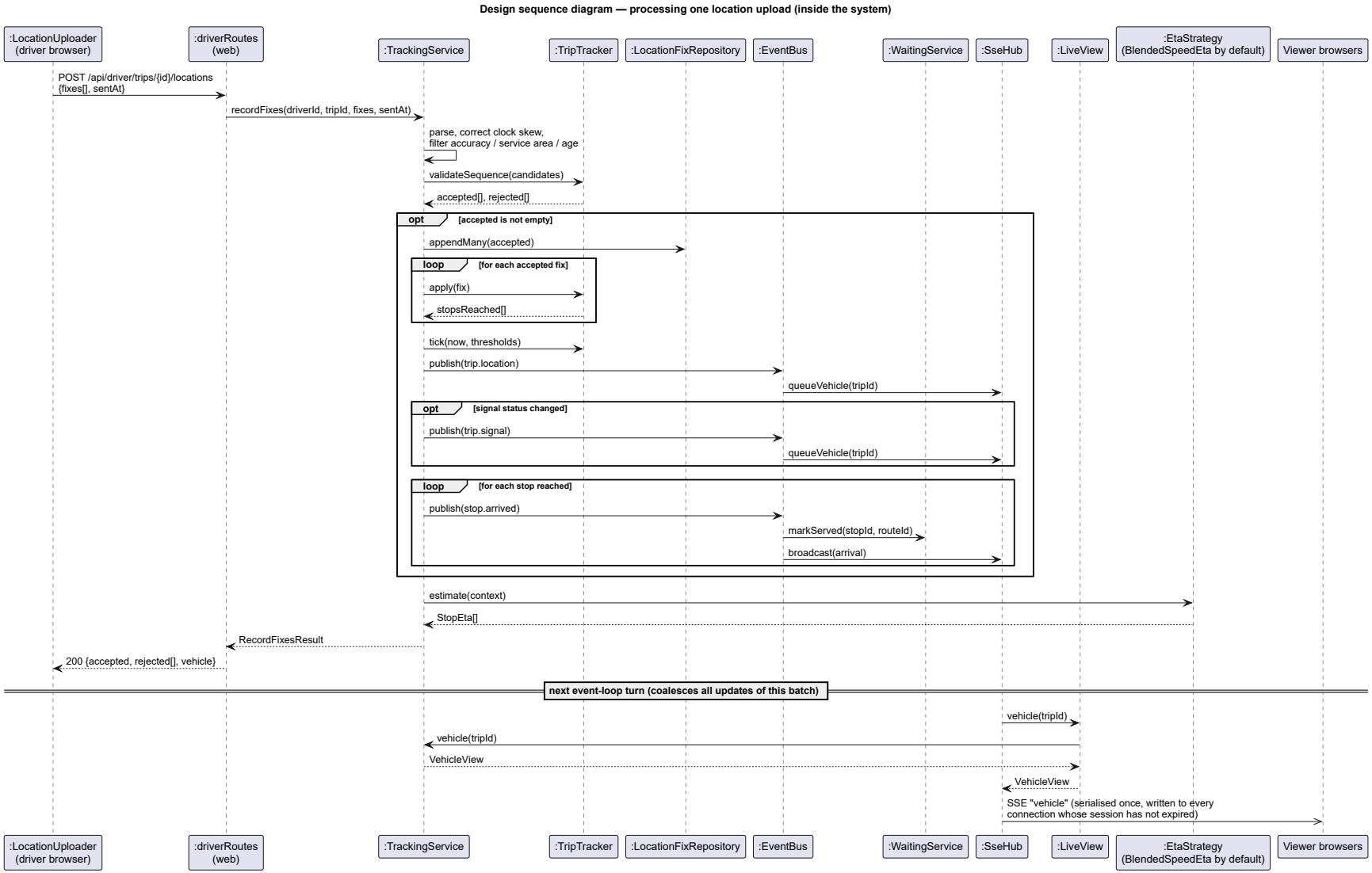


Figure A.5 — Inside the system: one location upload — overview at 49 % (read the 2 detail pages that follow, at 67 %; on screen, zoom in)

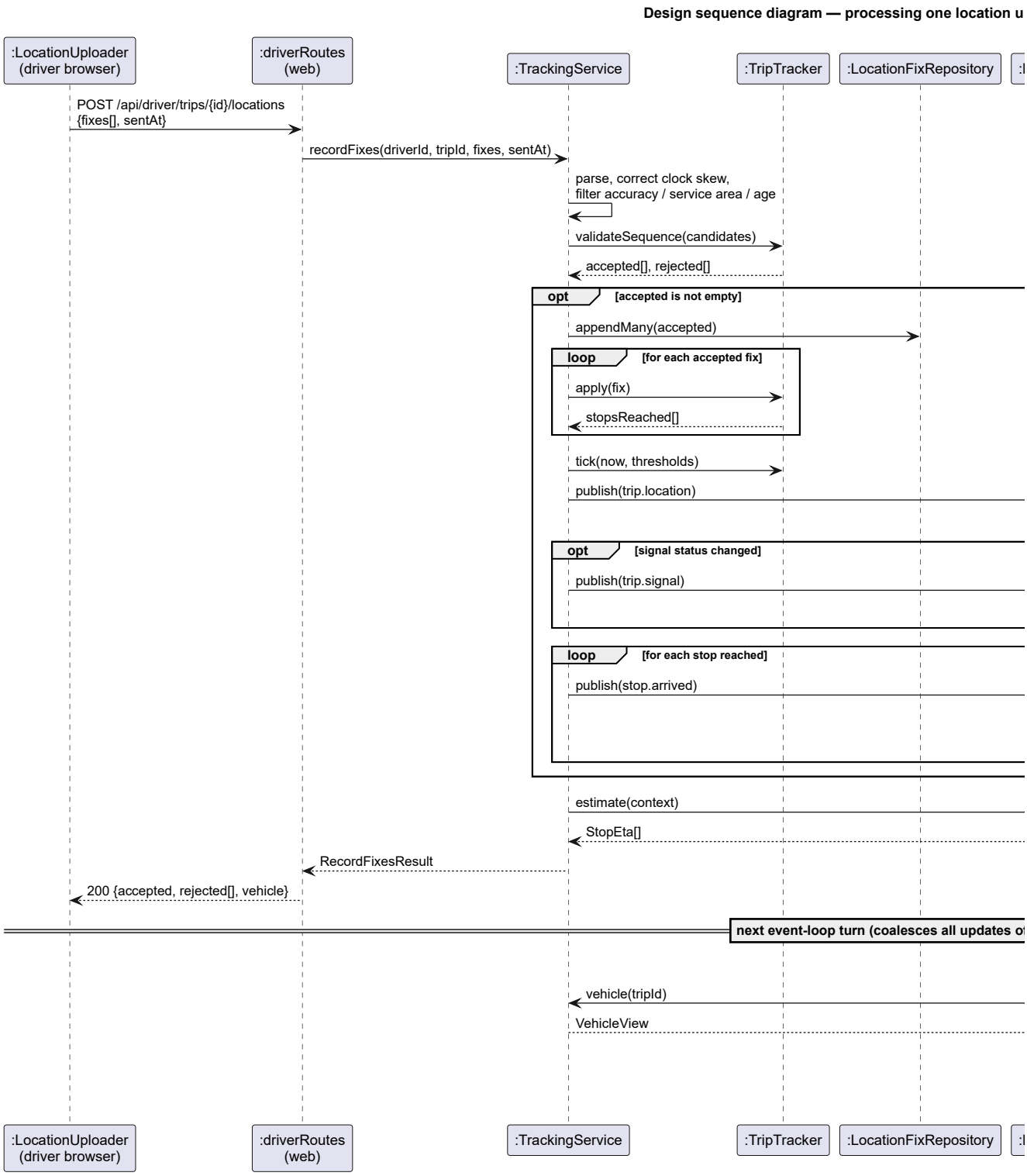


Figure A.5 — Inside the system: one location upload — detail 1 of 2 (column 1 of 2), at 67 %

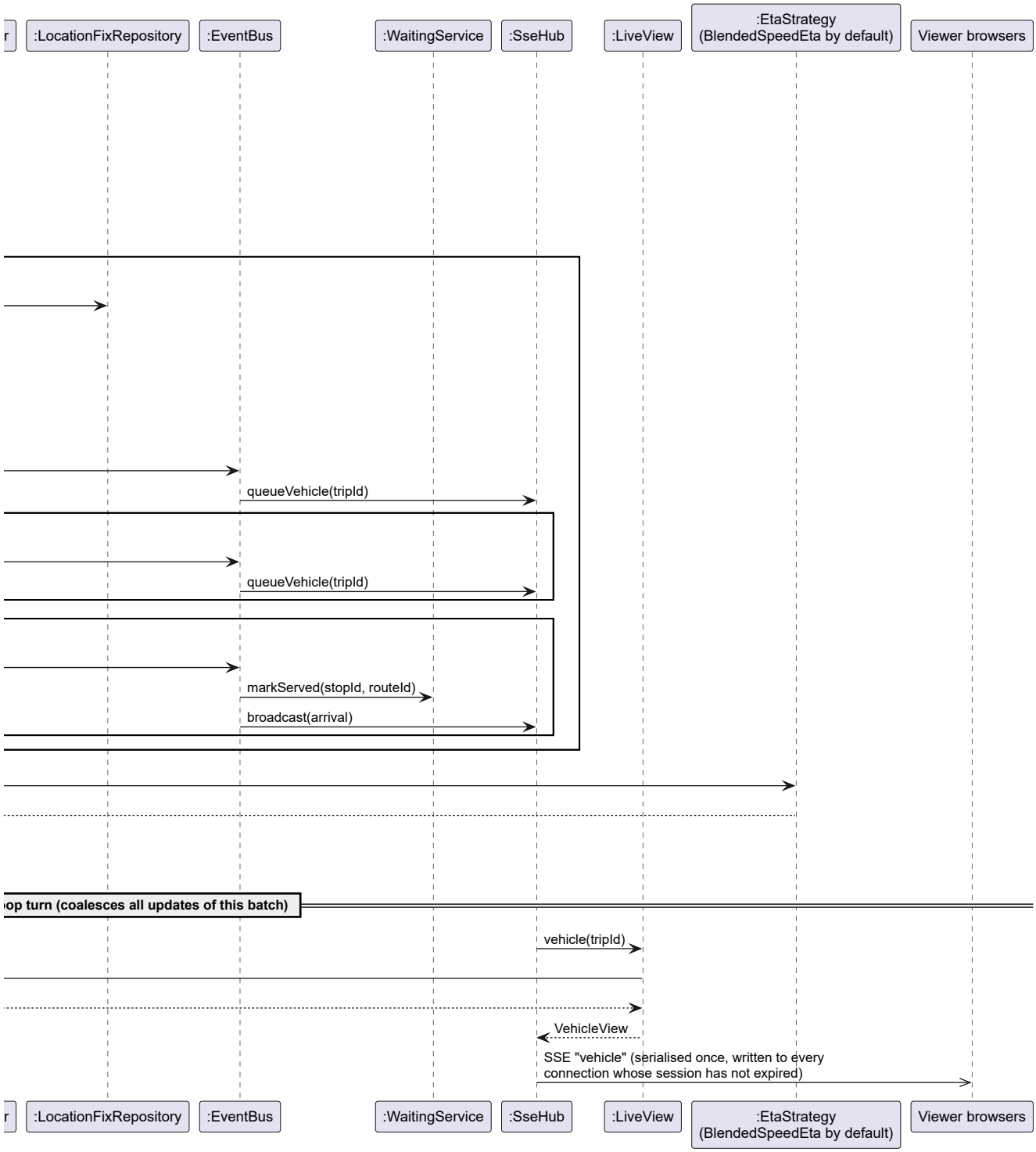


Figure A.5 — Inside the system: one location upload — detail 2 of 2 (column 2 of 2), at 67 %

A.6 State machine — Trip and GPS signal

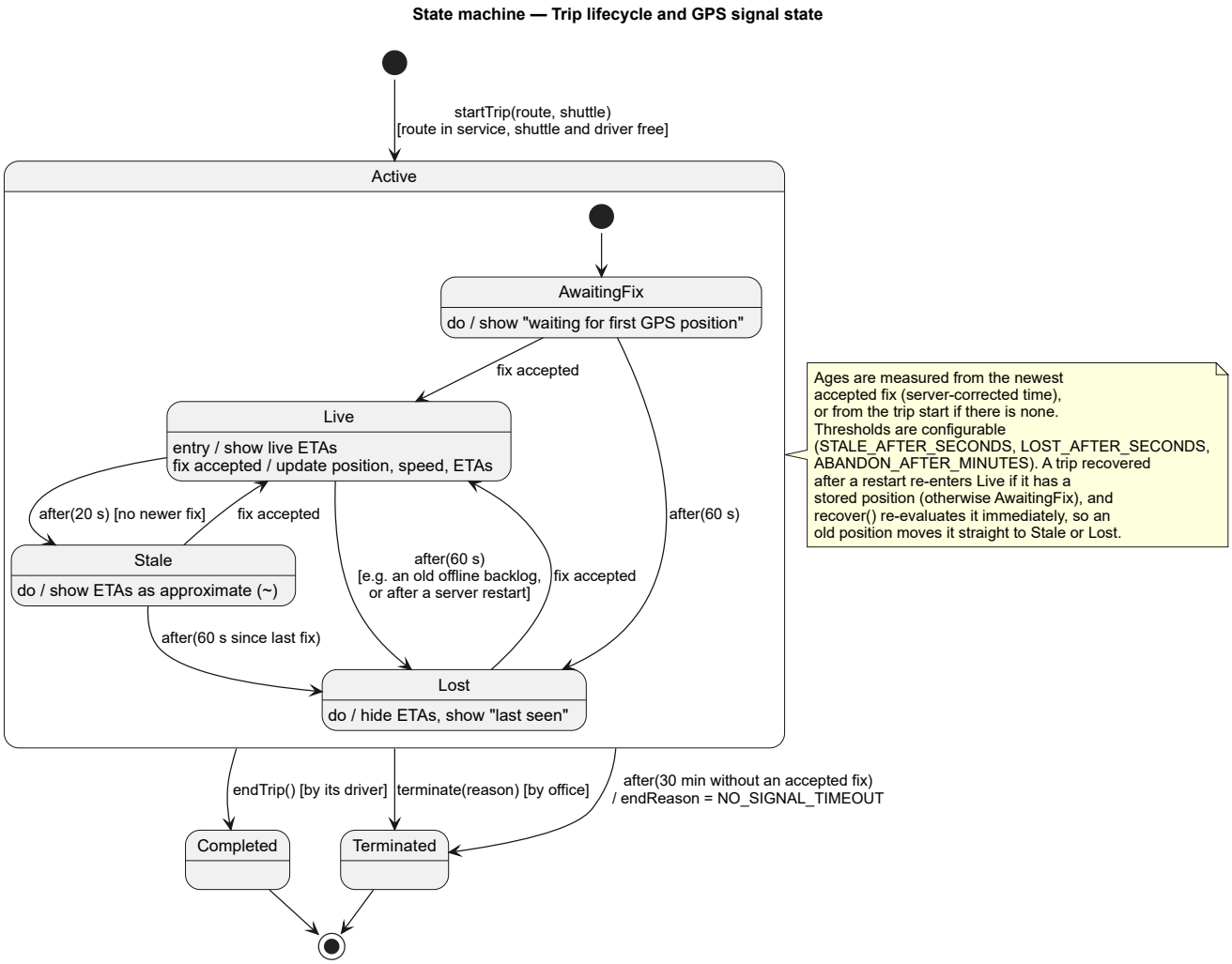


Figure A.6 — State machine for a trip and its GPS signal

`Active` is a composite state; its sub-states are implemented one-to-one by the State pattern classes in `src/tracking/signal-state.ts` (Figure A.15). Time events use UML `after(...)` notation.

A.7 State machine — Waiting request

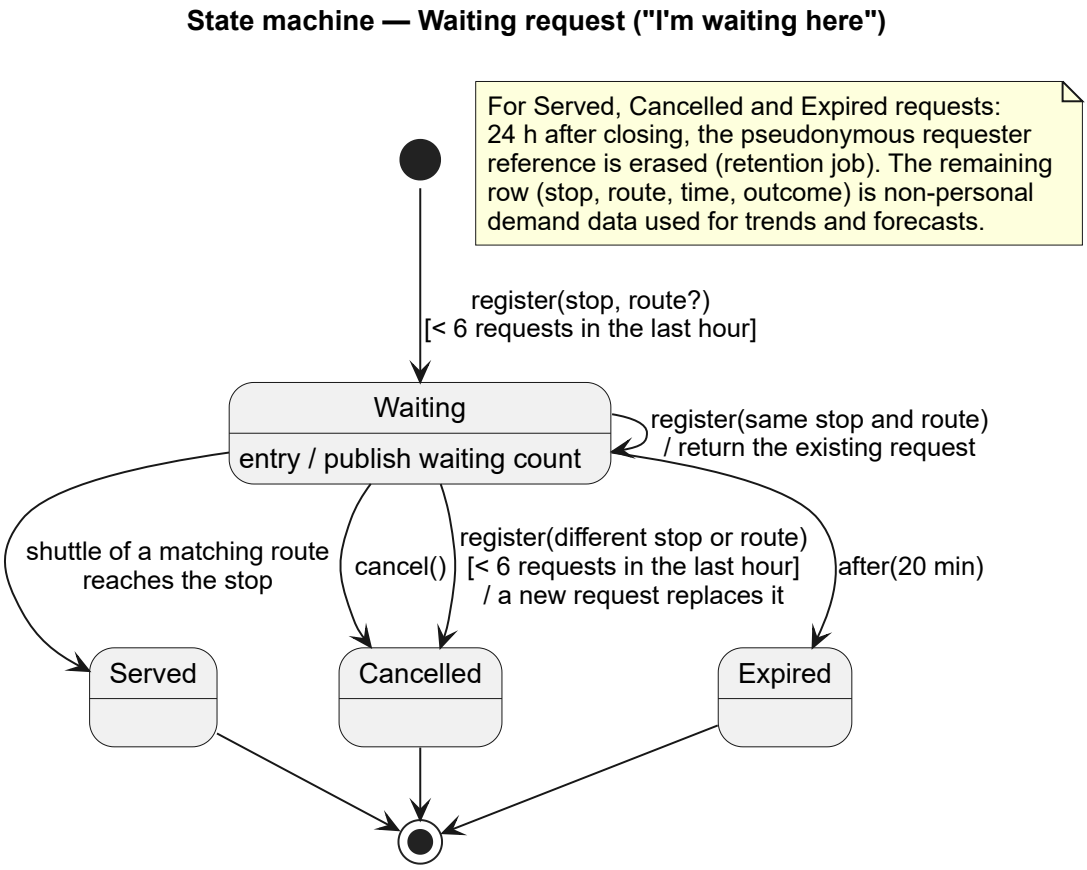


Figure A.7 — State machine for a waiting request

A.8 Domain model

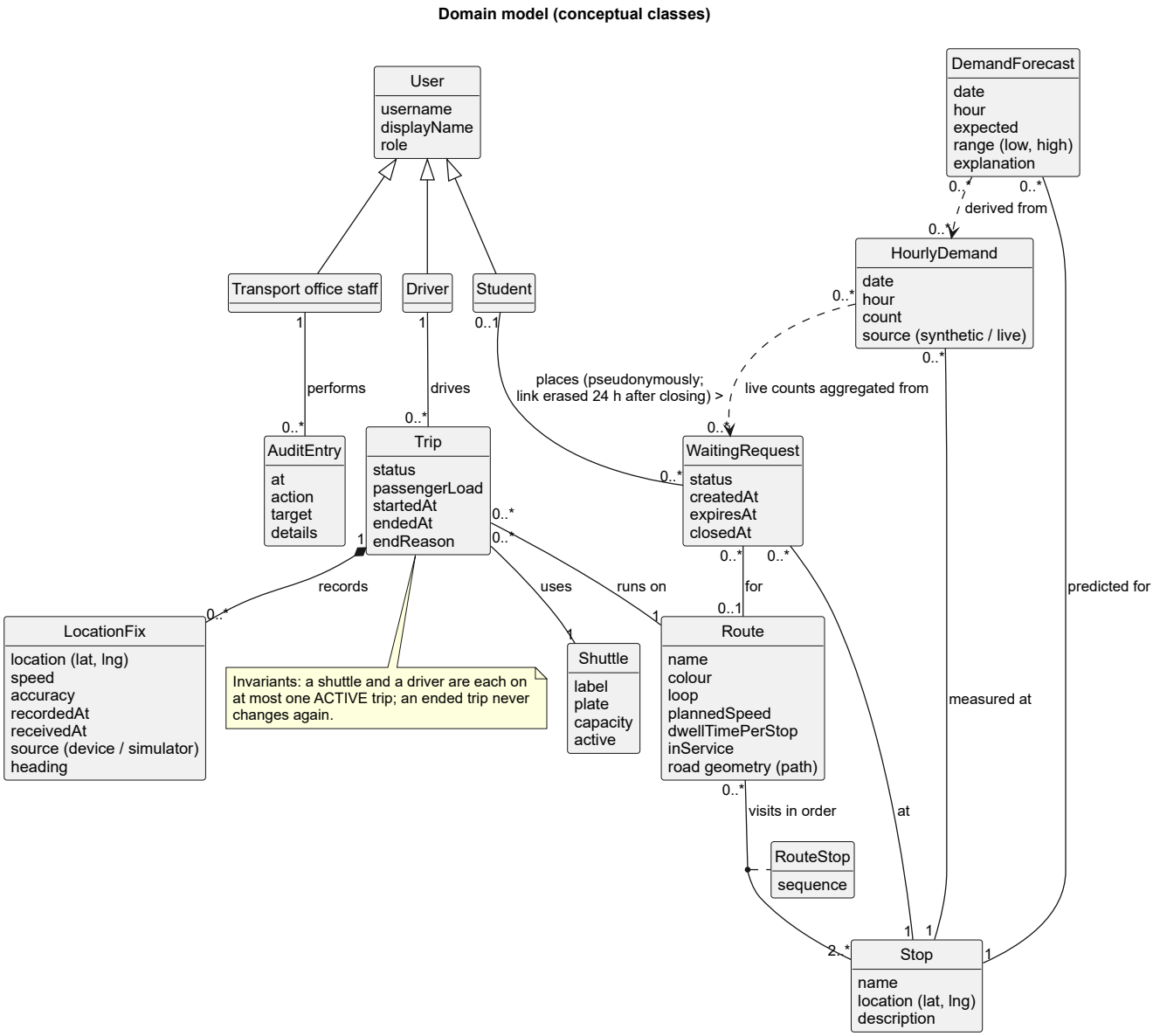


Figure A.8 — Domain model

Conceptual classes only (no operations). `RouteStop` is an association class carrying the stop order of a route.

A.9 Design class diagram 1 of 2 — tracking, network, real-time

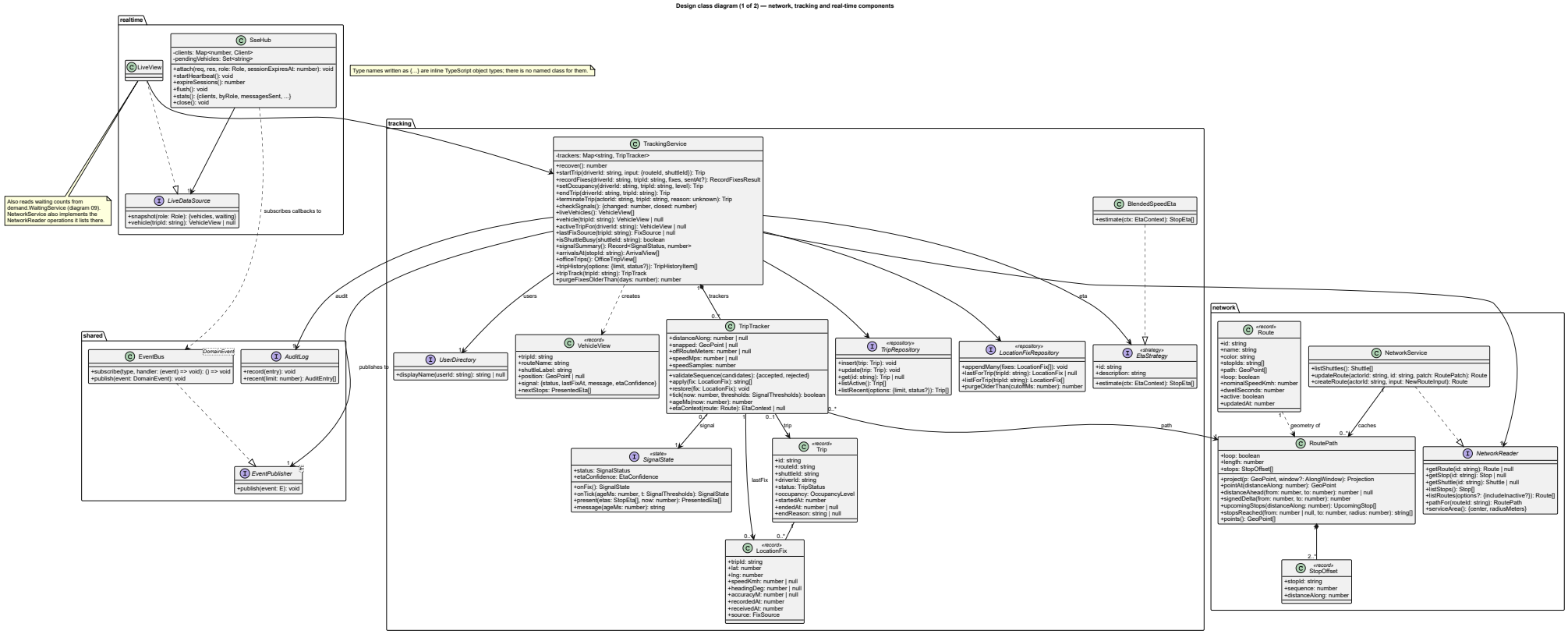


Figure A.9 — Design class diagram: tracking, network and real-time — overview at 28 % (read the 4 detail pages that follow, at 63 %; on screen, zoom in)

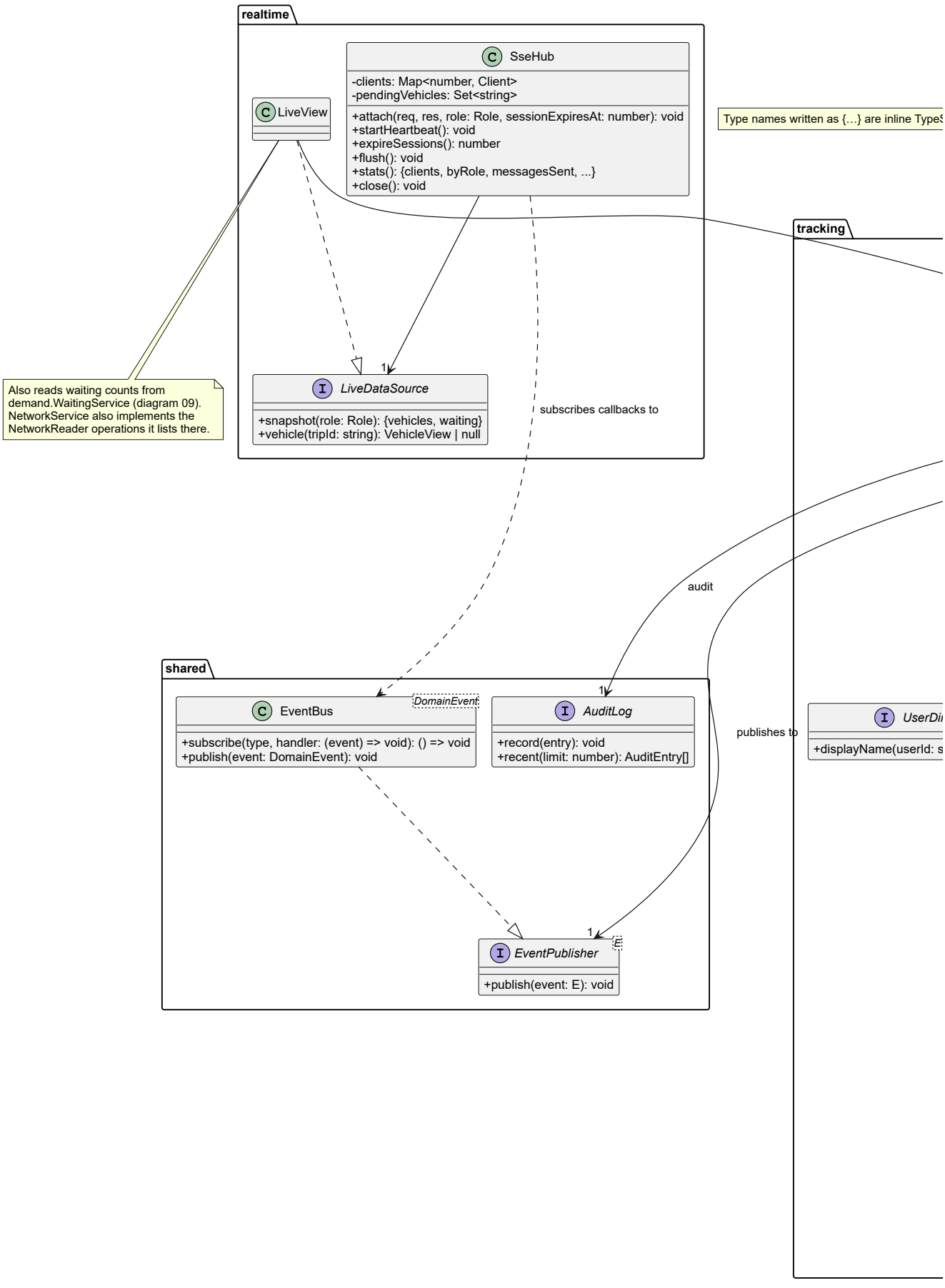


Figure A.9 — Design class diagram: tracking, network and real-time — detail 1 of 4 (column 1 of 4), at 63 %

mes written as {…} are inline TypeScript object types; there is no named class for them.

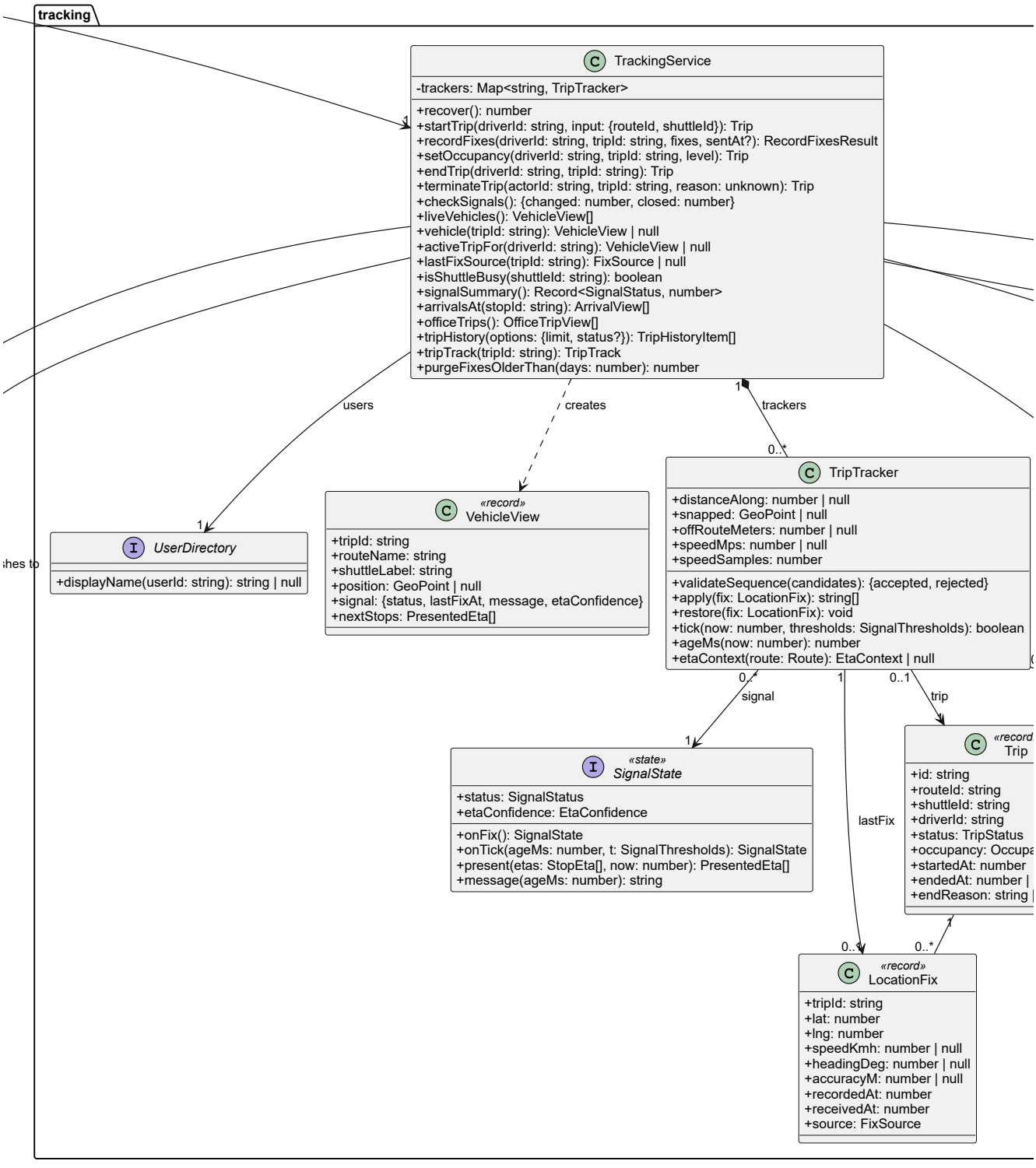


Figure A.9 — Design class diagram: tracking, network and real-time — detail 2 of 4 (column 2 of 4), at 63 %

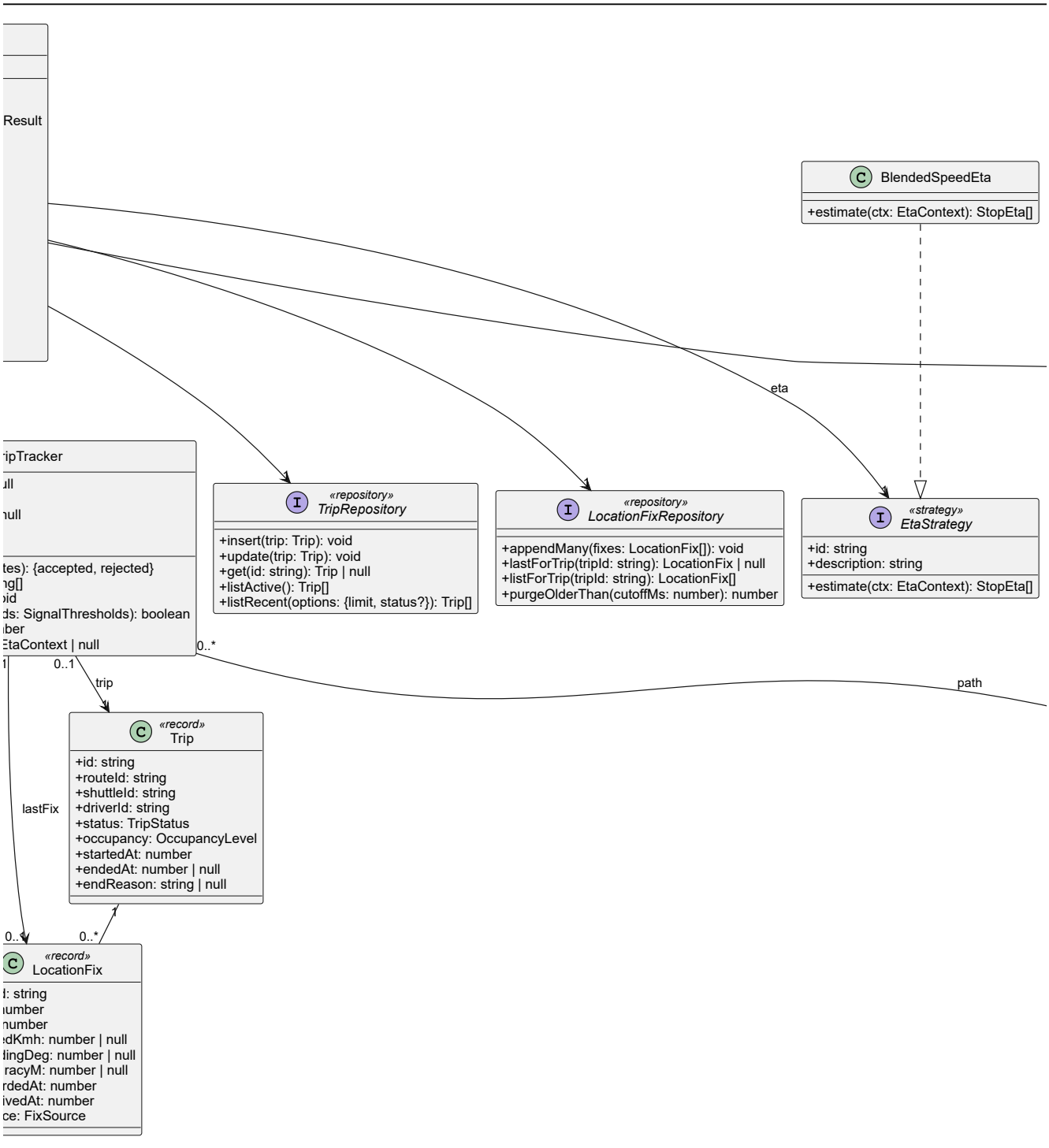


Figure A.9 — Design class diagram: tracking, network and real-time — detail 3 of 4 (column 3 of 4), at 63 %

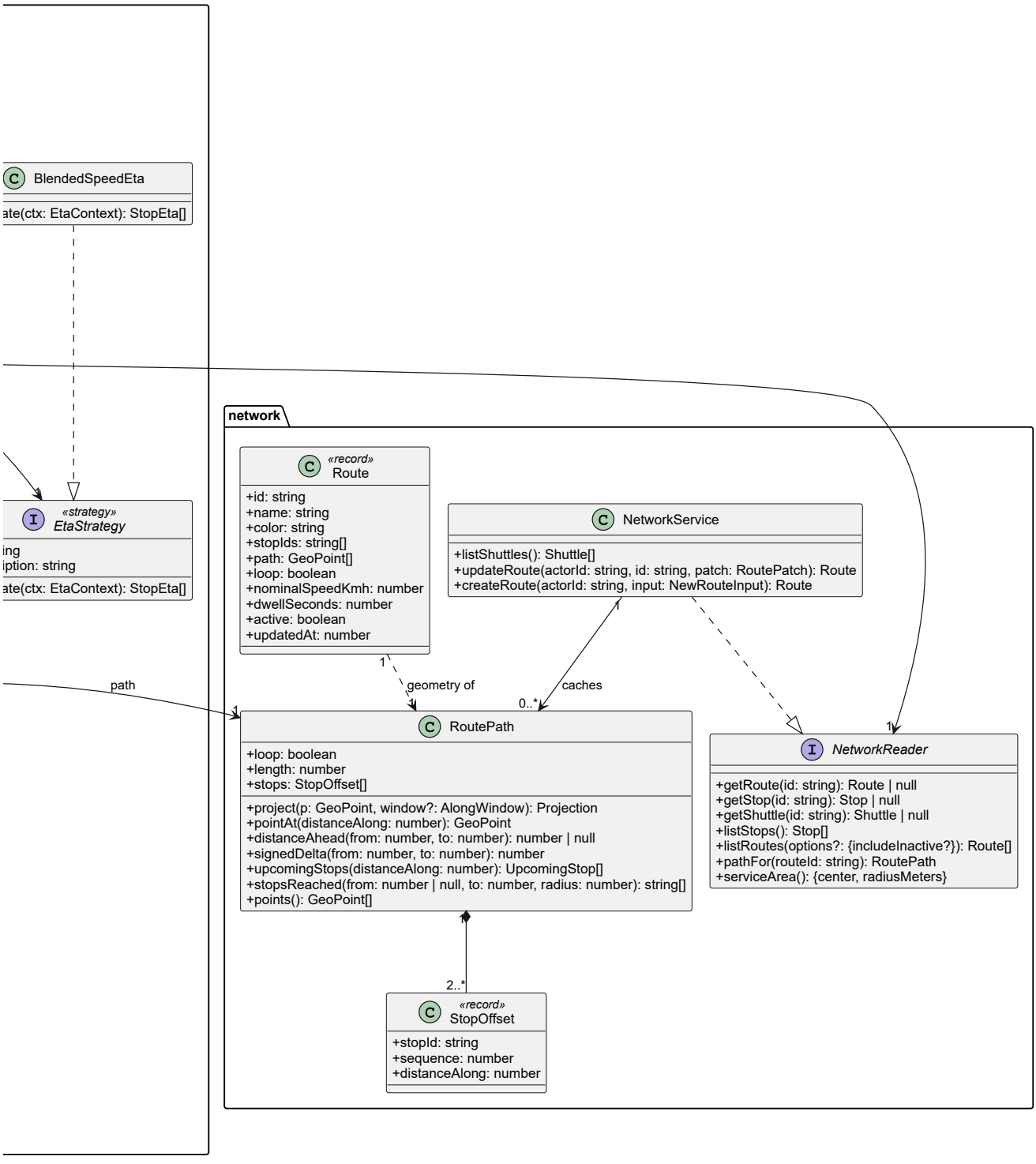


Figure A.9 — Design class diagram: tracking, network and real-time — detail 4 of 4 (column 4 of 4), at 63 %

A.10 Design class diagram 2 of 2 — demand, forecasting, identity

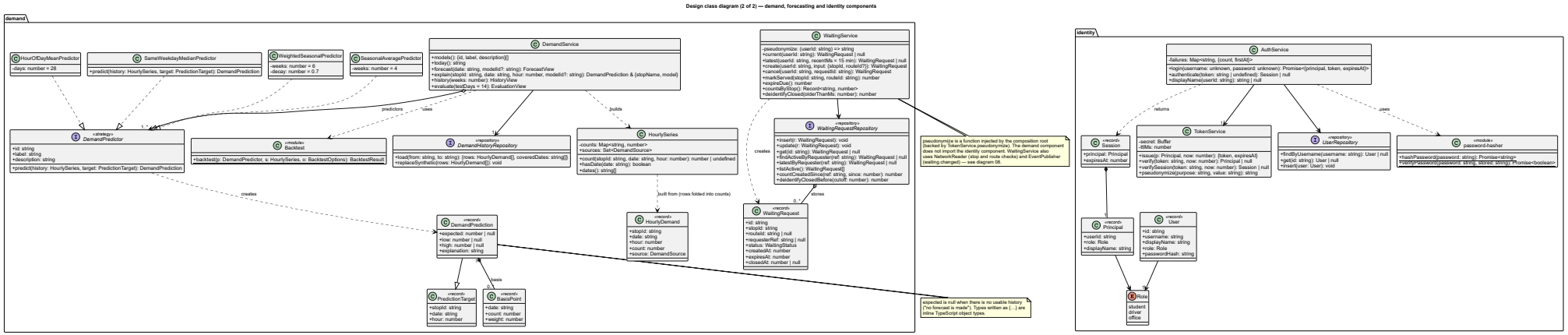


Figure A.10 — Design class diagram: demand, forecasting and identity — overview at 24 % (read the 3 detail pages that follow, at 65 %; on screen, zoom in)

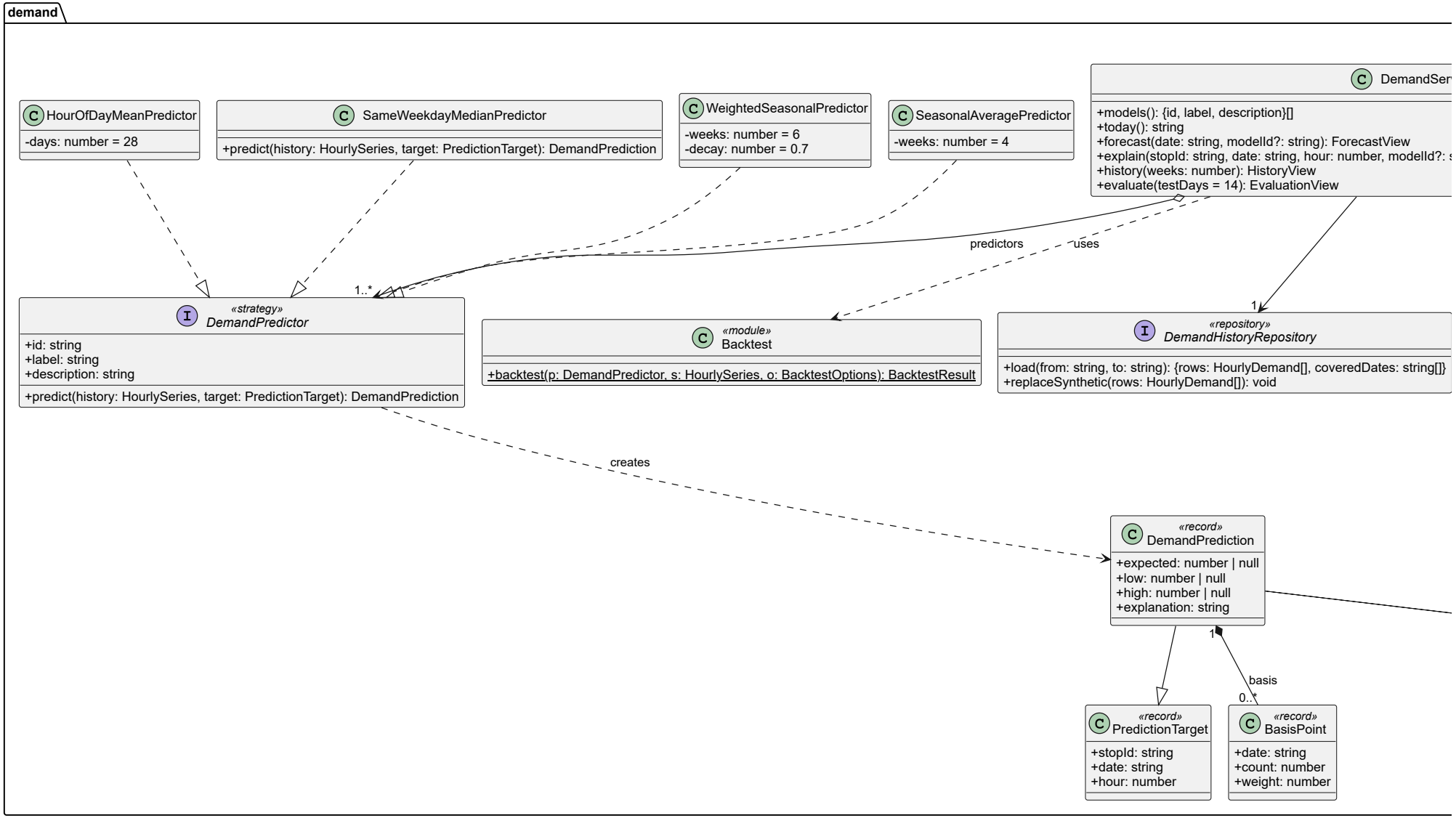


Figure A.10 — Design class diagram: demand, forecasting and identity — detail 1 of 3 (column 1 of 3), at 65 %

Design class diagram (2 of 2) — demand, forecasting and identity components

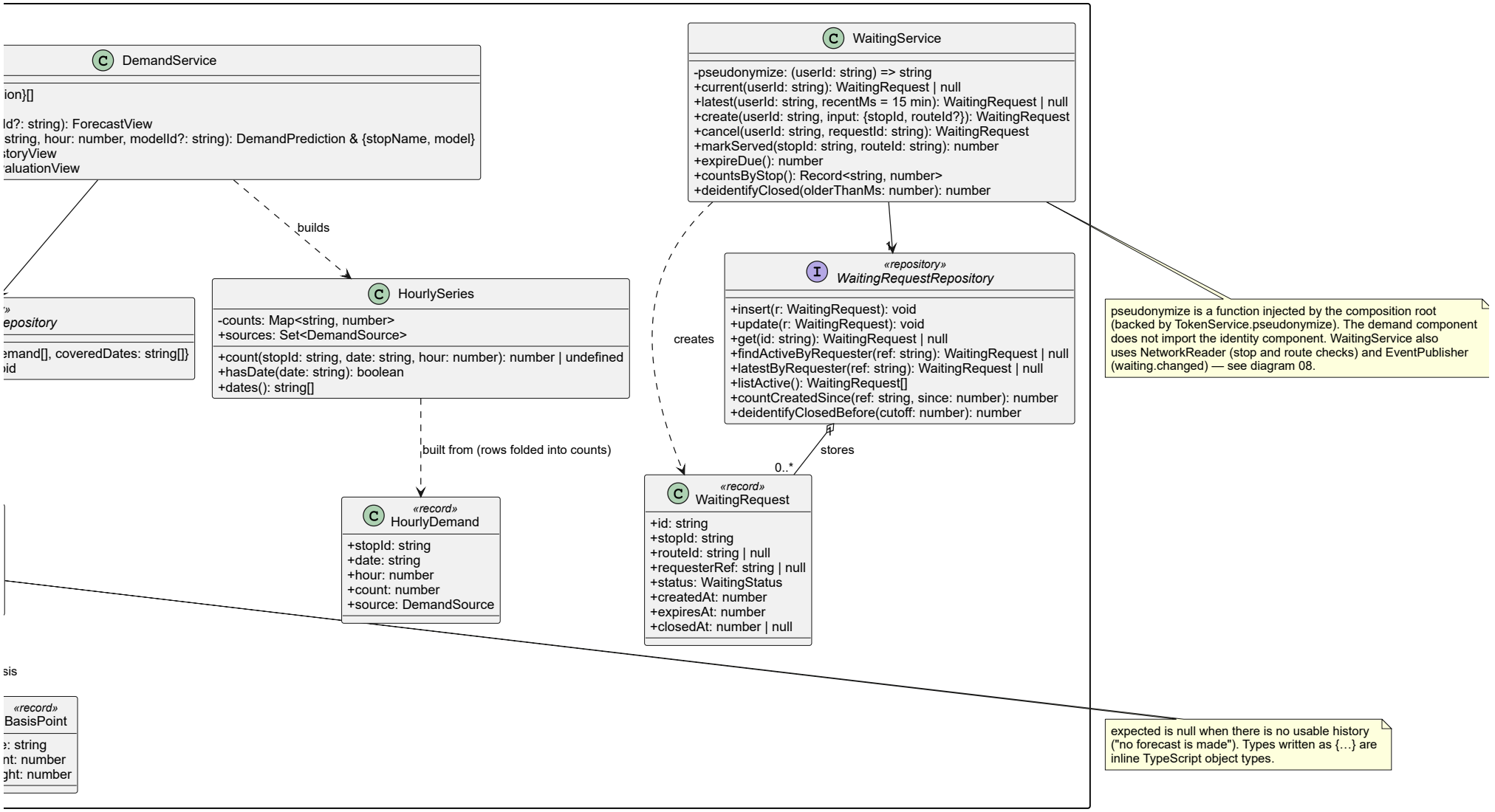


Figure A.10 — Design class diagram: demand, forecasting and identity — detail 2 of 3 (column 2 of 3), at 65 %

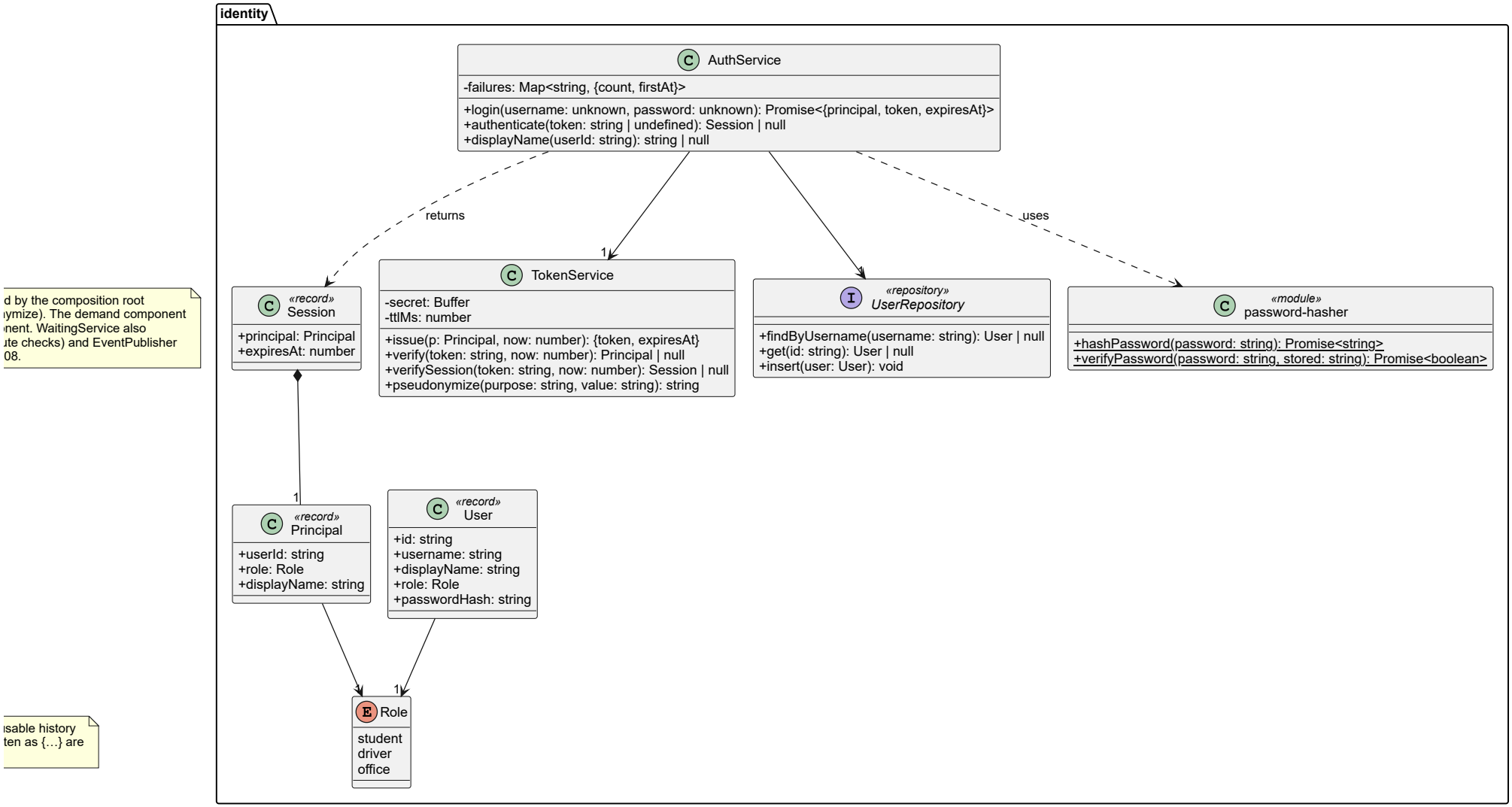


Figure A.10 — Design class diagram: demand, forecasting and identity — detail 3 of 3 (column 3 of 3), at 65 %

A.11 Component diagram

Each component is a folder under `src/` with a public service class and port interfaces. Provided interfaces are drawn as lollipops, required ones as dependencies on them.

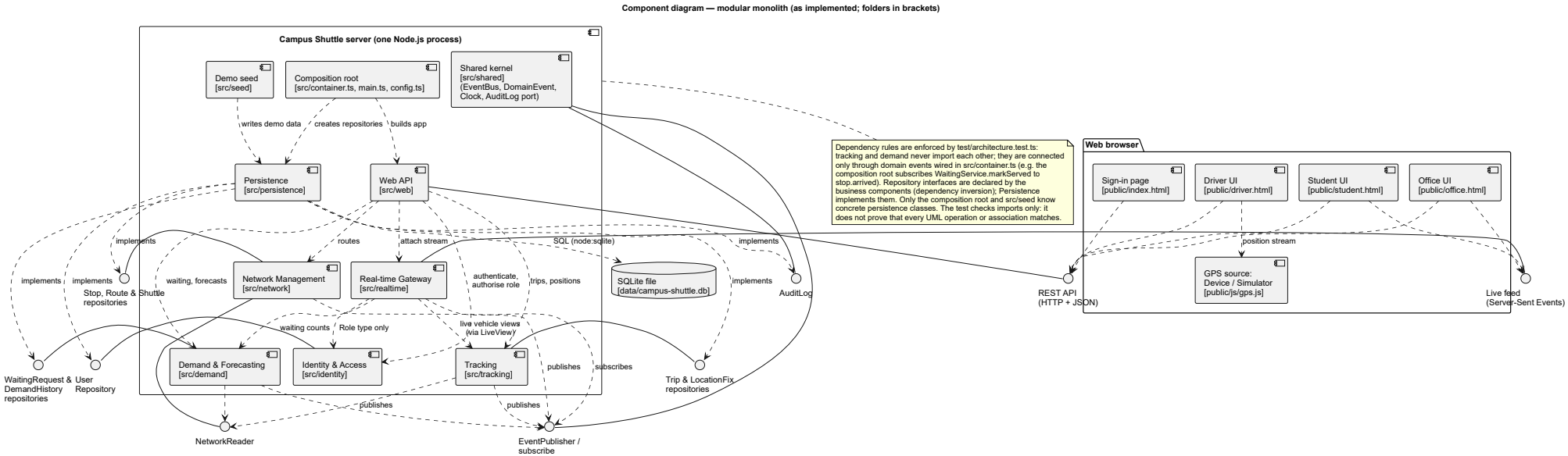


Figure A.11 — Component diagram — overview at 39 % (read the 2 detail pages that follow, at 72 %; on screen, zoom in)

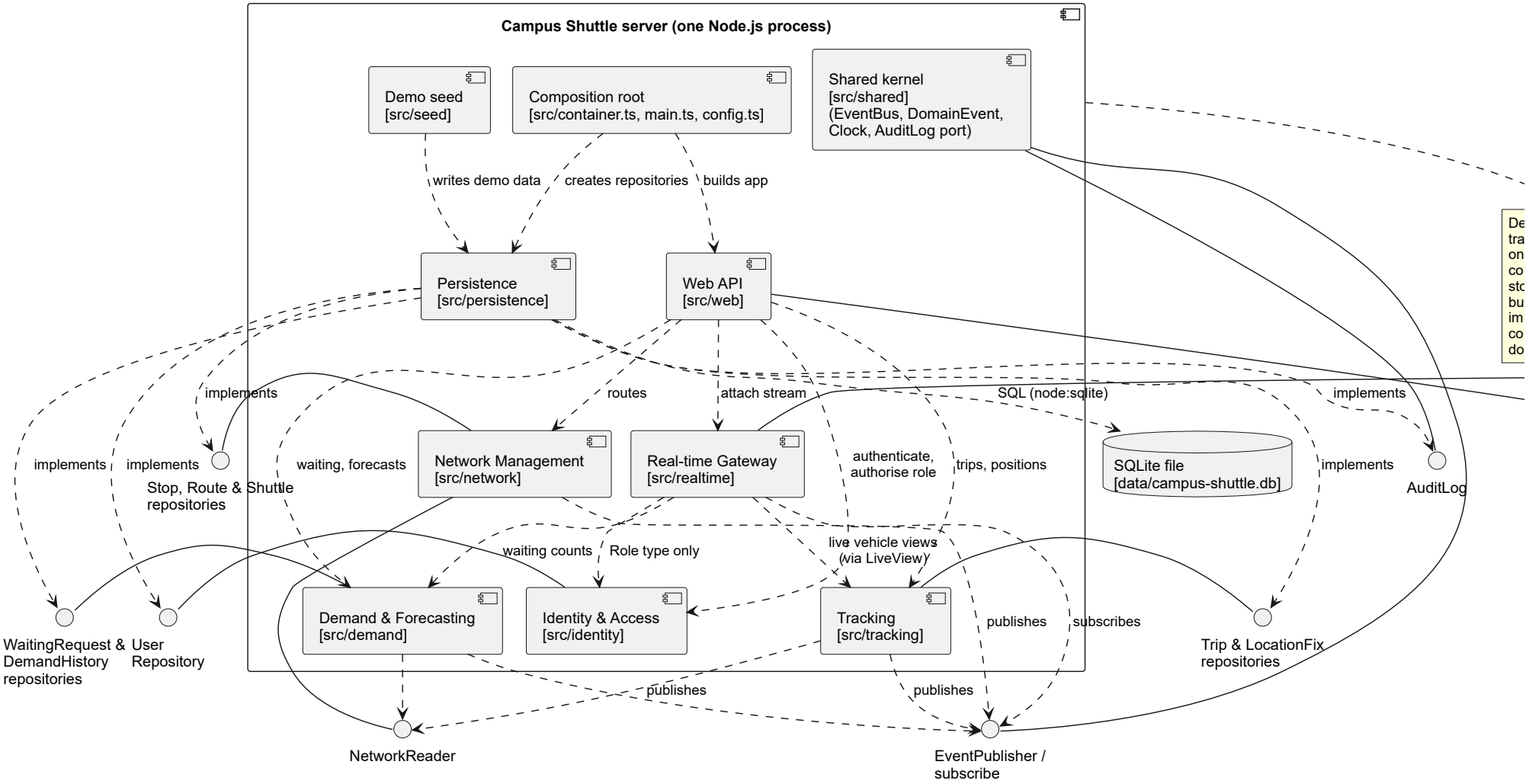


Figure A.11 — Component diagram — detail 1 of 2 (column 1 of 2), at 72 %

ıodular monolith (as implemented; folders in brackets)

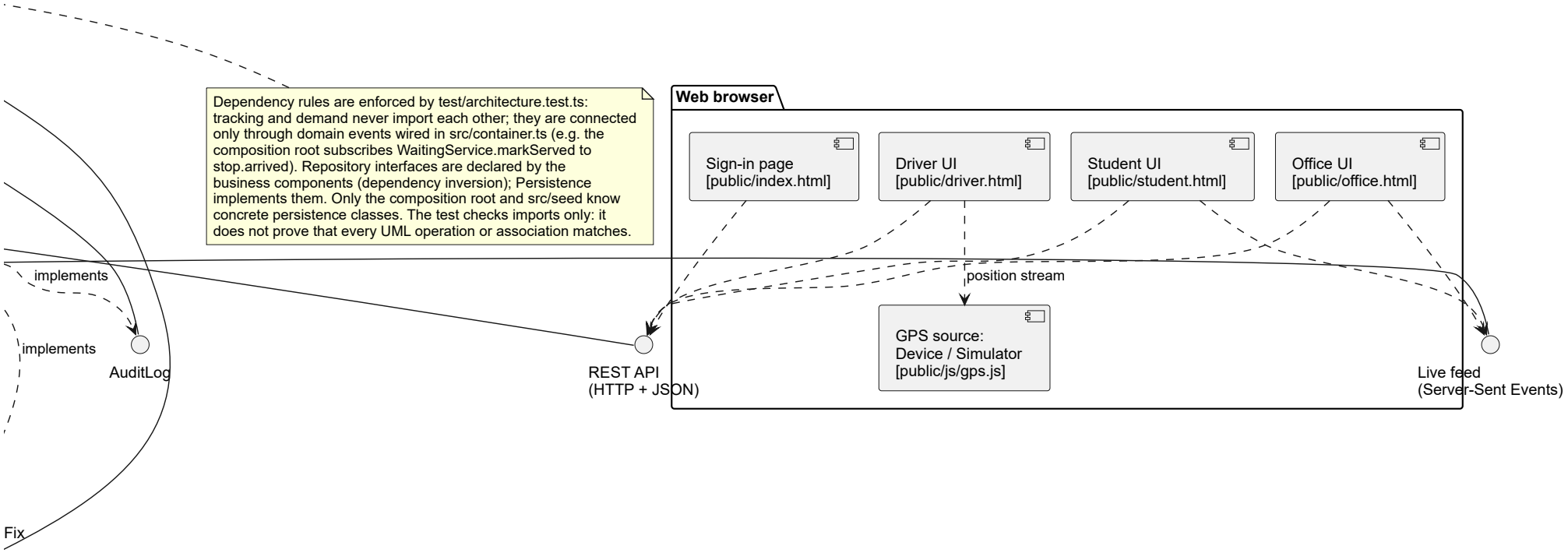


Figure A.11 — Component diagram — detail 2 of 2 (column 2 of 2), at 72 %

A.12 Deployment diagram

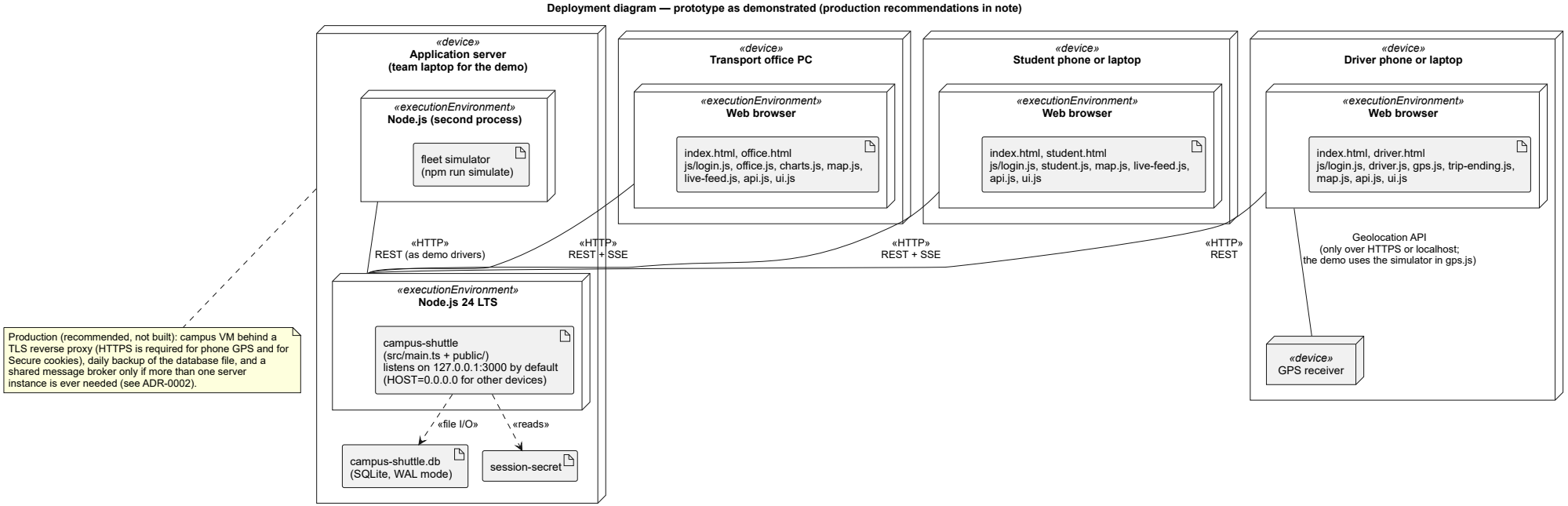


Figure A.12 — Deployment diagram — overview at 51 % (read the 2 detail pages that follow, at 93 %; on screen, zoom in)

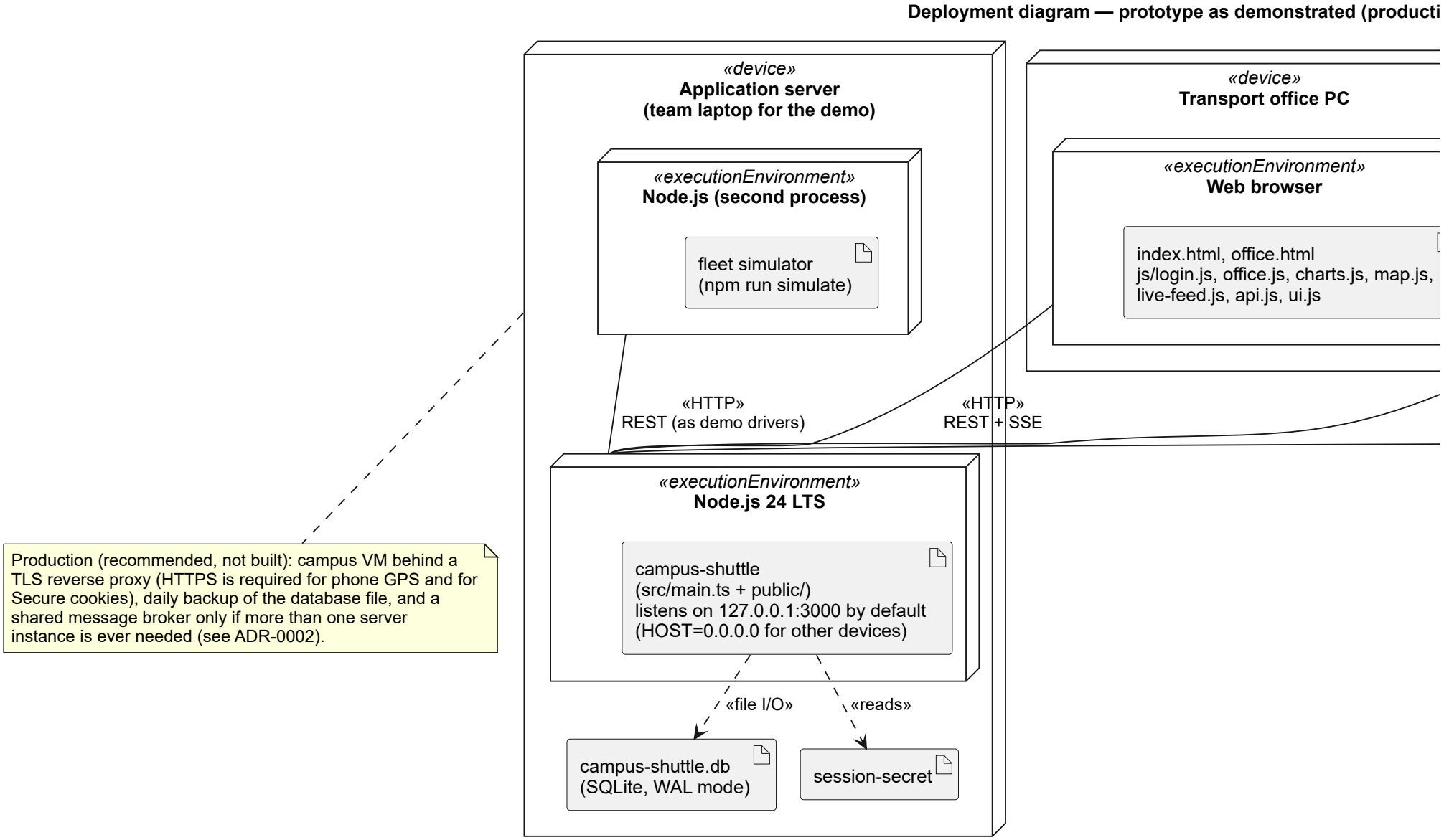


Figure A.12 — Deployment diagram — detail 1 of 2 (column 1 of 2), at 93 %

ype as demonstrated (production recommendations in note)

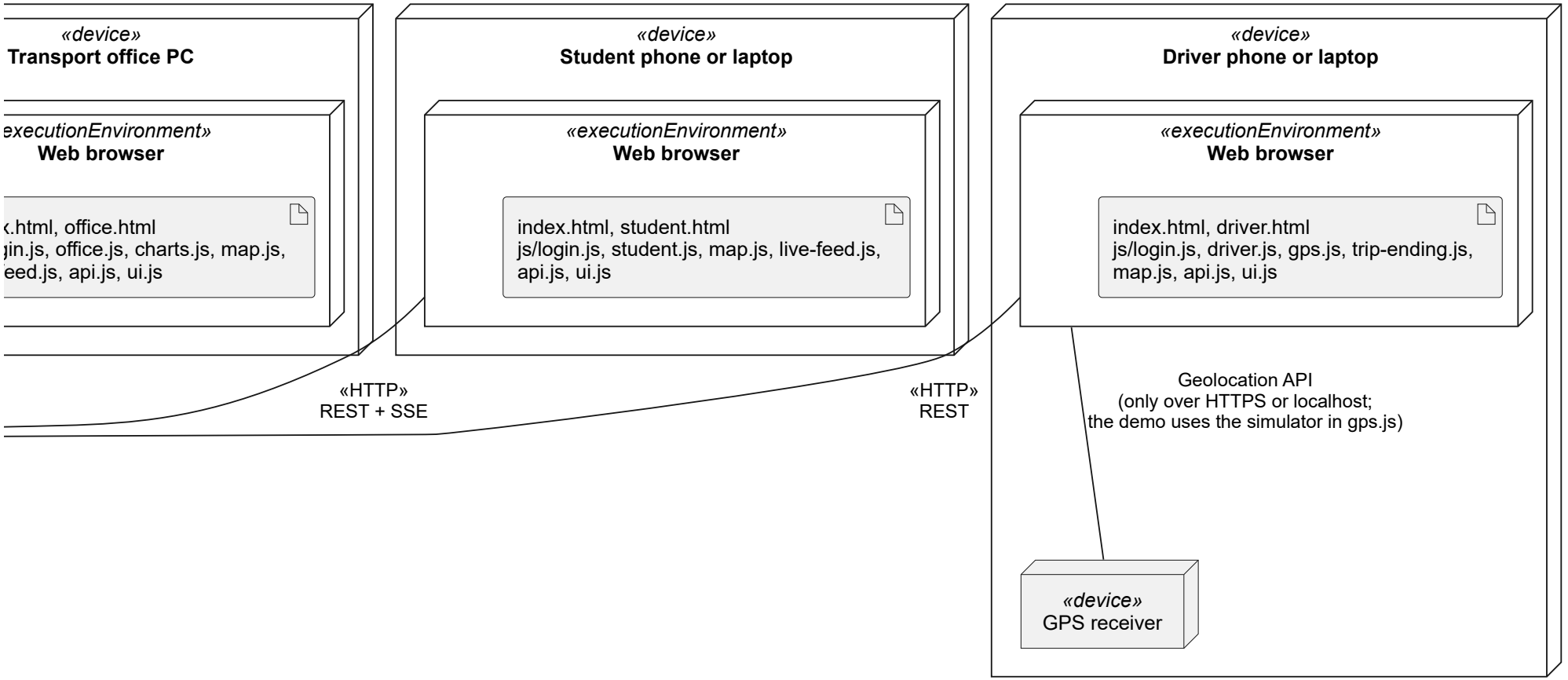


Figure A.12 — Deployment diagram — detail 2 of 2 (column 2 of 2), at 93 %

A.13–A.17 Design pattern class diagrams

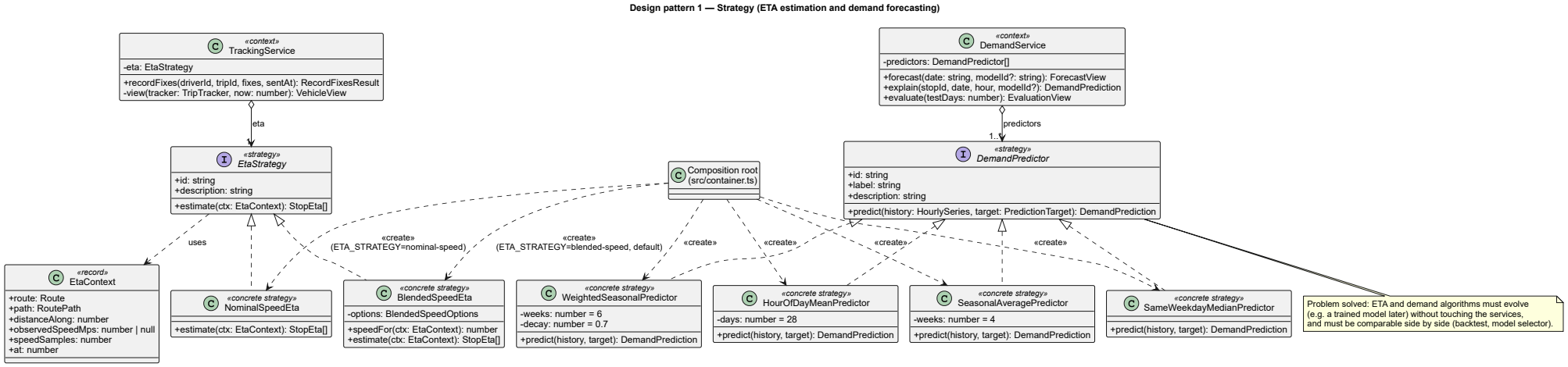


Figure A.13 — Strategy pattern — overview at 43 % (read the 2 detail pages that follow, at 79 %; on screen, zoom in)

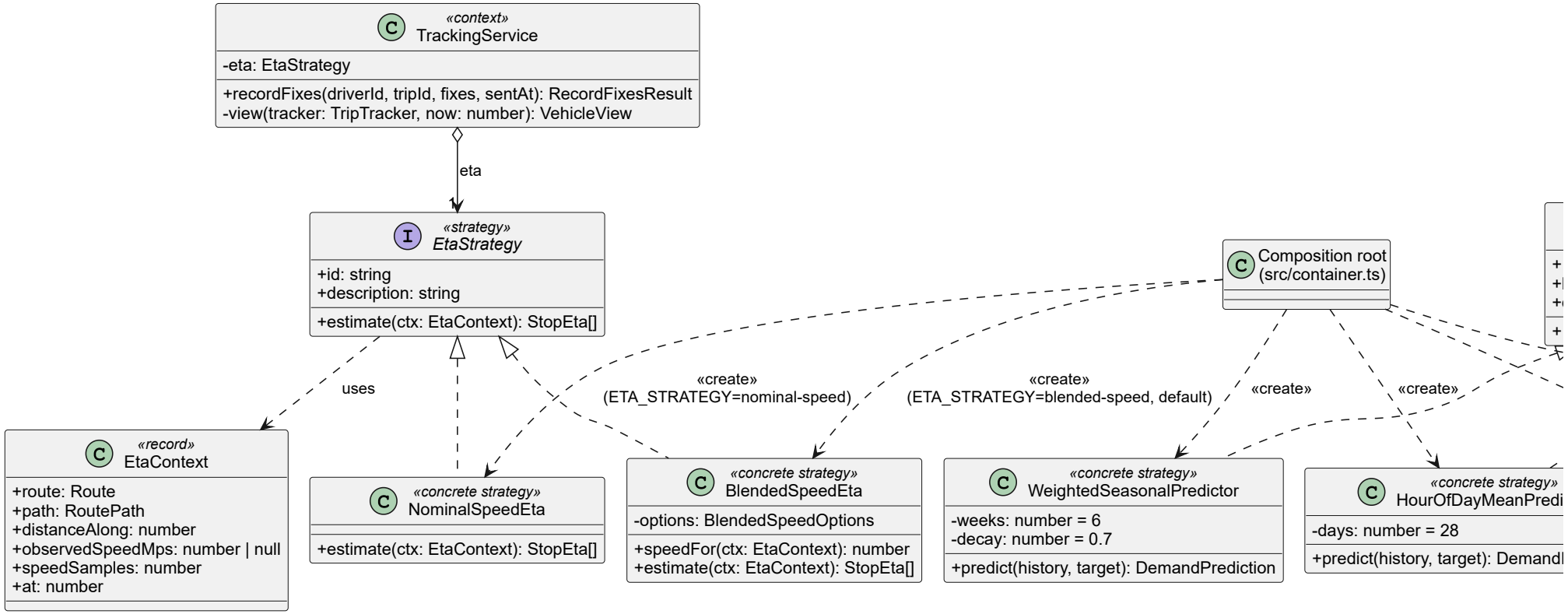


Figure A.13 — Strategy pattern — detail 1 of 2 (column 1 of 2), at 79 %

Strategy (ETA estimation and demand forecasting)

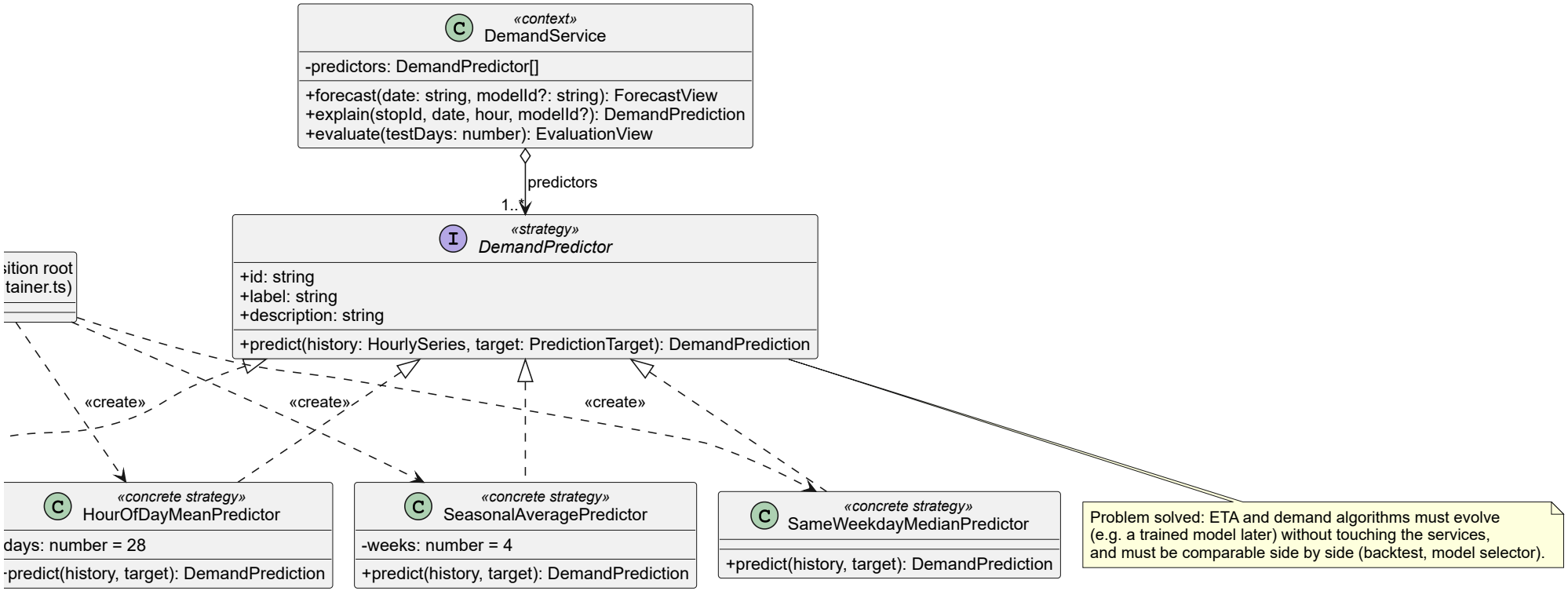


Figure A.13 — Strategy pattern — detail 2 of 2 (column 2 of 2), at 79 %

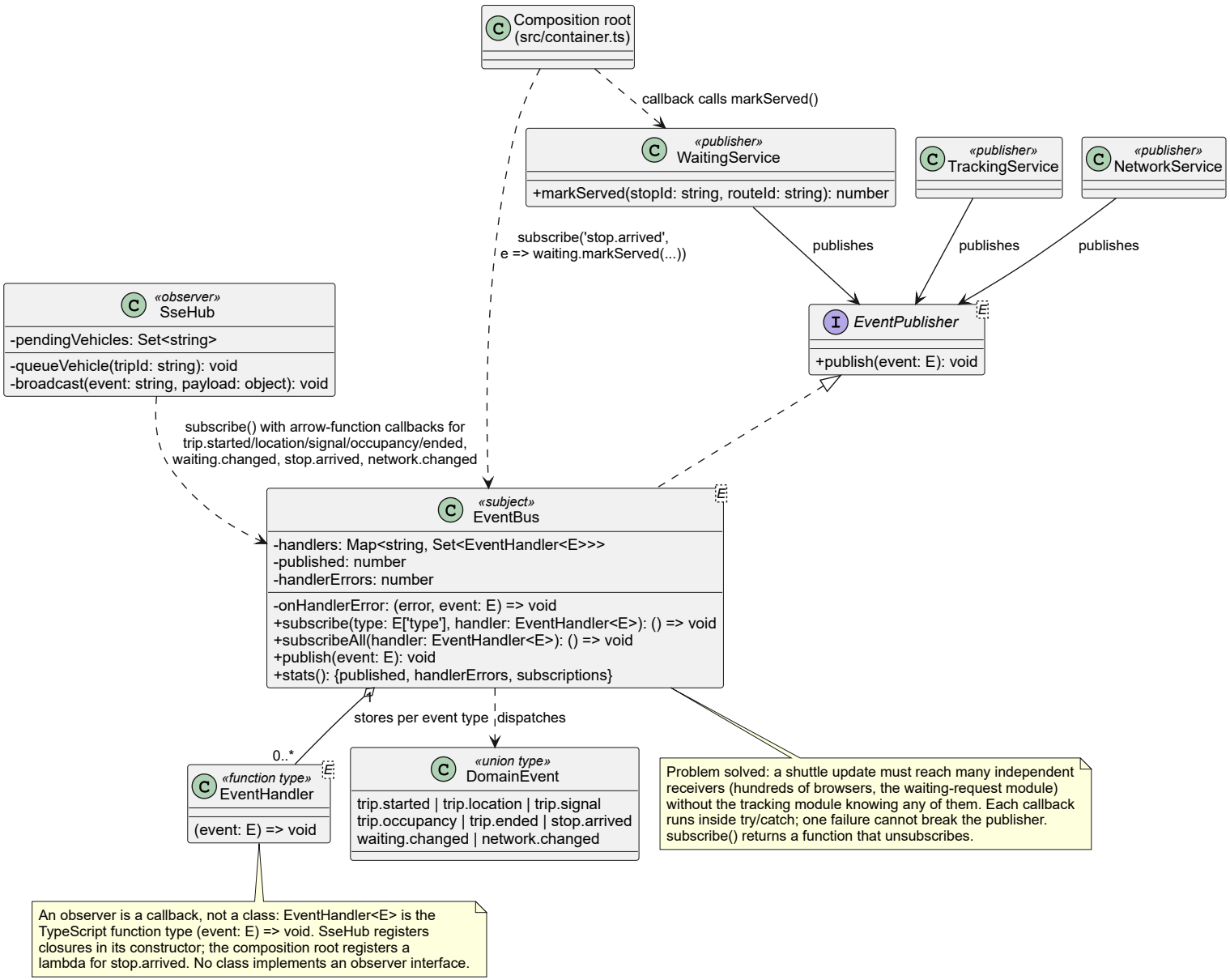


Figure A.14 — Observer pattern

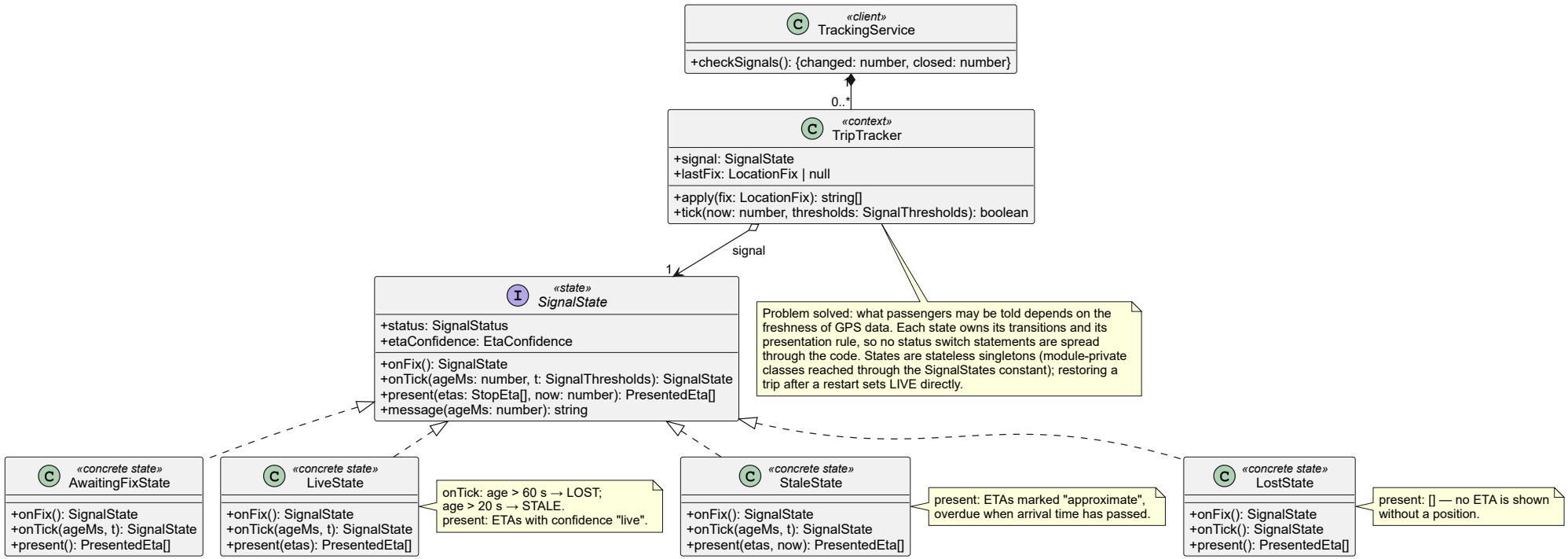


Figure A.15 — State pattern

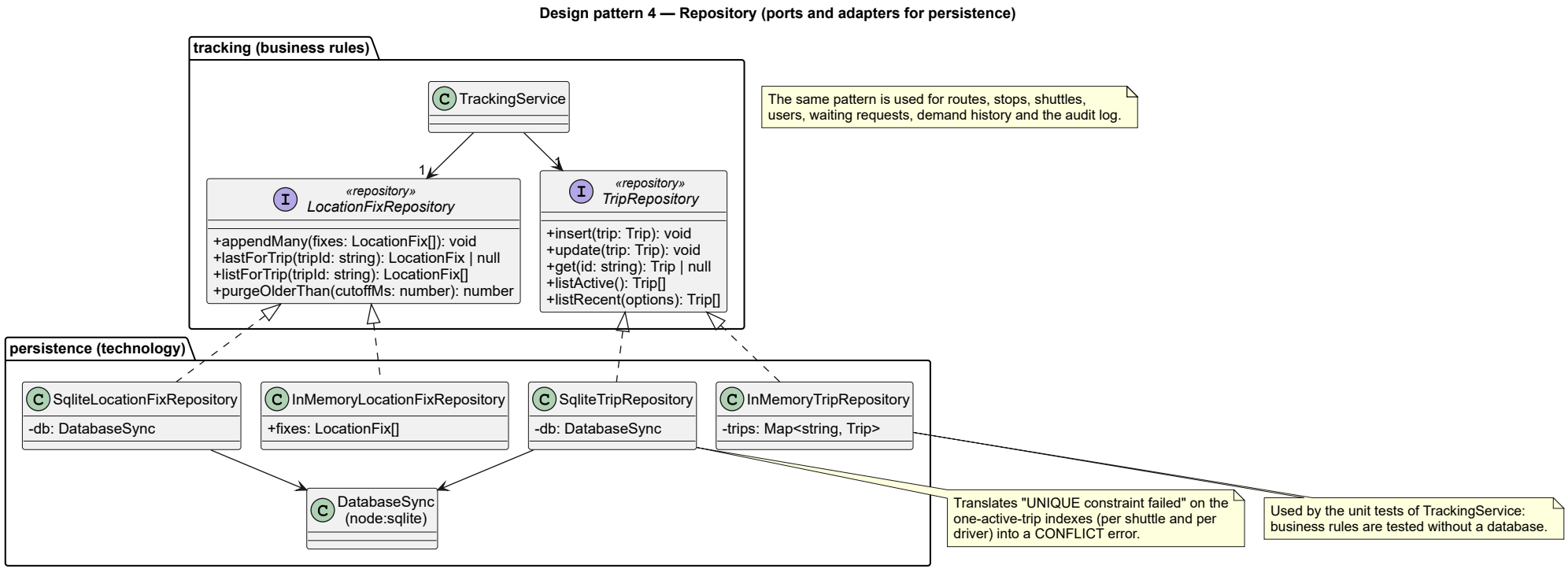


Figure A.16 — Repository pattern

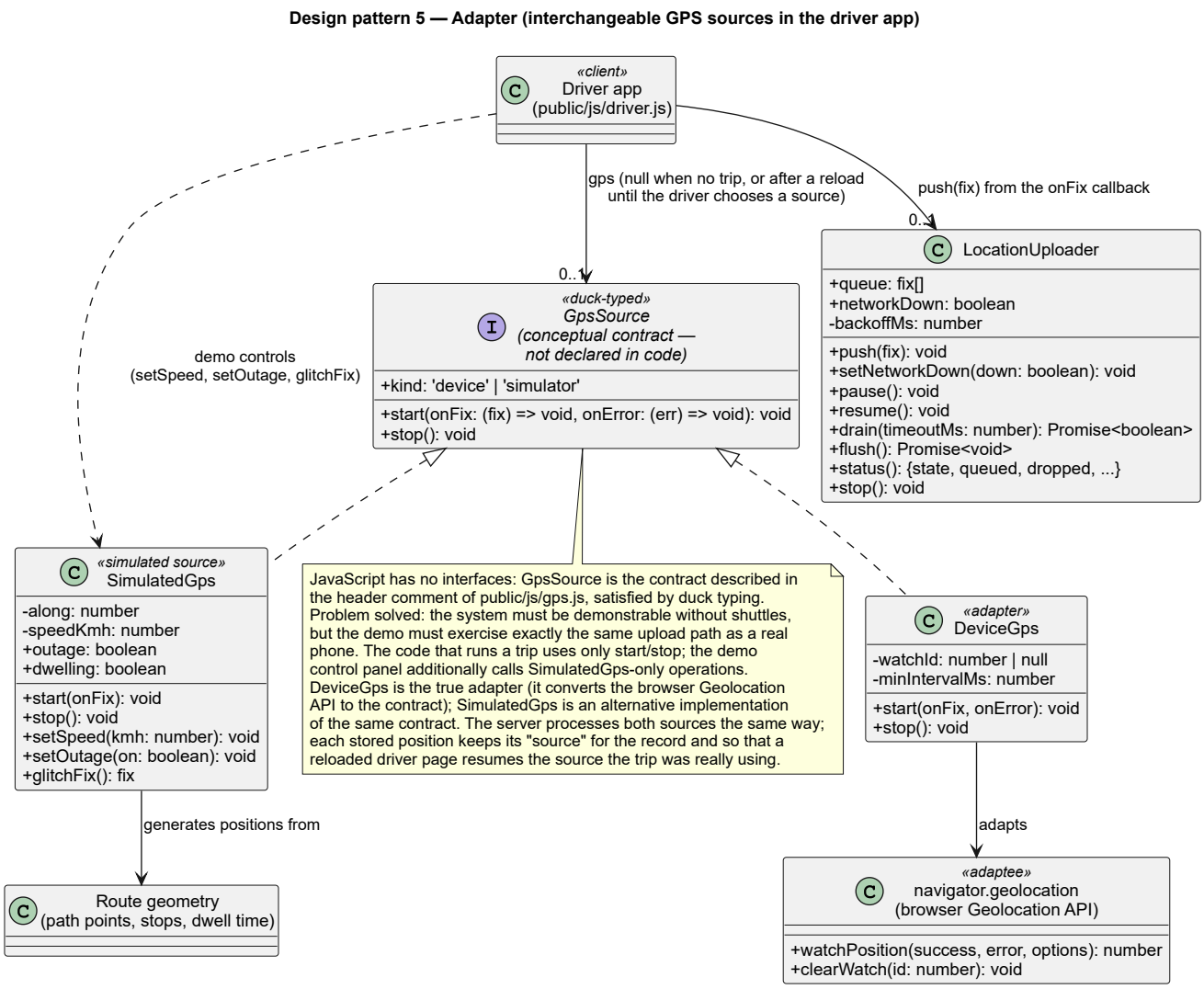


Figure A.17 — Adapter pattern